

# Adaptive BT Generator Platform — 동작 원리 설명서

기반 문서: ADAPTIVE\_BT\_PLAN.md v7.0  
목적: 기본 아이디어, 설계 구조, 동작 방식을 수학 표기 + 자연어로 정리

## 0. 핵심 용어 사전 (처음 읽는 사람을 위한 빠른 참조)

| 용어                | 풀네임/의미                                                                                                                 |
|-------------------|------------------------------------------------------------------------------------------------------------------------|
| BT                | Behavior Tree. 전투기 AI의 행동 결정 구조. "거리가 1500ft 이하이면 → GunAttack 실행" 같은 조건-행동 규칙들을 트리 구조로 연결한 것. 프로그래머가 직접 설계하는 규칙 기반 시스템 |
| EIM               | Enemy Intent Model. 적의 현재 의도(CLOSING/GUN_RUN 등)를 관측 시계열로 분류하는 신경망 모델                                                   |
| BFM               | Basic Fighter Maneuvers. 공중전의 기본 기동. 공격(OBFM), 방어(DBFM), 정면교전(HABFM) 3종류로 분류됨                                          |
| WR                | Win Rate. 승률 (0~100%)                                                                                                  |
| HP                | Hit Points. 피격 체력. 줄어들면 불리, 0이 되면 격추                                                                                   |
| pp                | percentage point. 퍼센트 포인트. 83%→100%이면 +17pp                                                                            |
| tick              | 시뮬레이터의 최소 시간 단위. 이 시스템에서 1 tick = 0.2초                                                                                 |
| $\mathbb{R}^n$    | n차원 실수 공간. $\mathbb{R}^{28}$ 은 "28개의 실수로 이루어진 벡터"라는 뜻                                                                  |
| 정책 함수 $\pi$       | Policy function. "현재 상황(관측값)을 보고 다음 행동을 결정하는 함수". 인간 조종사의 판단에 해당                                                       |
| 임베딩 (Embedding)   | 고차원 데이터(20×28=560개 수치)를 저차원(64개 수치)으로 압축한 벡터. 유사한 패턴은 임베딩 공간에서 가까이 위치함                                                 |
| 프로토타입 (Prototype) | 각 클래스의 "대표 벡터". 해당 클래스 학습 샘플들의 임베딩 평균값                                                                                 |
| YAML              | Yet Another Markup Language. 사람이 읽기 쉬운 설정 파일 형식. BT 구조를 텍스트로 기술하는 데 사용                                                 |
| Wilson CI         | Wilson Score Interval. 승률의 신뢰구간. 예: "100매치 WR 60% → 95% 신뢰구간 ±10%"                                                     |

## 1. 왜 이 시스템이 필요한가 (핵심 동기)

### 문제: 정적 BT(Static Behavior Tree)의 한계

**BT (Behavior Tree)란?** 전투기 AI의 행동 결정 구조다. "거리가 1500ft 이하이고 ATA가 12° 이하이면 → 사격"처럼 조건(Condition)과 행동(Action)을 트리 구조로 연결한 규칙 모음이다. 프로그래머가 직접 설계하며, 규칙이 고정되어 있어 "정적(Static)"이라고 부른다.

단일 BT를 만들어서 모든 상대에게 쓰면 어떻게 될까? 실험에서 직접 확인됐다.

#### H-E 계열 실험 결과 (4 variants, 전부 기각)

H-E: Energy Conversion 로직을 변경한 BT 변형 실험. 4가지 에너지 운용 방식을 각각 6명의 상대에게 테스트.

- 상대 A에게 같은 BT 수정 적용 → **+63 HP** (체력 증가 = 승률 상승)
- 상대 B에게 같은 BT 수정 적용 → **-17 HP** (체력 감소 = 승률 하락)

이것을 **Pareto trade-off**라고 한다. 한 상대에 대한 성능 개선이 다른 상대에 대한 성능 저하를 동반하는 상황 — 모든 상대를 동시에 개선하는 단일 전략은 존재하지 않는다.

이론적 근거: Jensen 부등식

$$\mathbb{E}_o \left[ \max_x f(x, o) \right]$$

Adaptive: 상대마다 최적 전략 선택

$$\geq \max_x \mathbb{E}_o [f(x, o)]$$

Static: 모든 상대에 평균 최적인 단일 전략

- $o$ : 상대(opponent)
- $x$ : 전략(BT)
- $f(x, o)$ : 전략  $x$ 로 상대  $o$ 를 만났을 때의 성능 (승률 등)

해석: "상대마다 최적 전략을 골라 쓴 평균 성능"은 항상 "단일 고정 전략의 성능"보다 같거나 낫다. Adaptive BT의 이론적 우위는 수학적으로 보장된다.

⚠ **Jensen 부등식의 전제 조건** — 이 보장은 "상대 식별이 정확하다"는 암묵적 가정 위에 성립한다. EIM 커버리지가 26.9%(§11.15.3, §5.5)인 구간에서는  $\arg \max_x$ 가 잘못된  $x$ 를 집어서 **보장이 붕괴**. 695폴에서 v9 WR 55%(§9)가 이론과 실측의 괴리를 보여준다. 즉:

- "adaptive  $\geq$  static" (강한 주장): intent 식별이 확률적으로 균등하게 정확할 때
- 실무 주장: **intent 식별 정확도  $\times$  커버리지가 임계값 이상일 때만** adaptive 우위
- 적 intent가 잘못 분류되는 구간에선 static BT가 더 안전할 수 있음 (§11.14 참조)

실험 검증:

| 방식                      | 6 opponents WR        | 695 폴 WR         | 비고                  |
|-------------------------|-----------------------|------------------|---------------------|
| Static BT (v6h2)        | 83.3%                 | 54.96%           | 경계선 — intent 식별 불필요 |
| <b>Adaptive BT (v9)</b> | <b>100%</b> (+16.7pp) | 55.01% (+0.05pp) | 6-opp에만 과적합 신호      |

695폴 차이가 오차범위 내라는 사실이 "coverage gap 73.1%에서 adaptive가 실제로 기여 못함"을 시사한다.

2. 한 문장 핵심 아이디어

적의 관측 궤적 패턴을 실시간 분류하고(L1), 각 패턴에 대해 가장 높은 승률을 기록한 BFM 기동을 선택하고(L2), 그 기동을 상대 방의 관측값에 따라 파라미터 수준에서 최적화한다(L3).

3-레이어 의사결정(§11.14):

- **L1 Classify**: 적 관측 시퀀스  $\rightarrow$  intent 클래스
- **L2 Select**: intent  $\times$  state bin  $\rightarrow$  BFM 기동 노드
- **L3 Optimize**: 노드 파라미터  $\theta$ 를 자기-적 관측으로 실시간 튜닝

**BFM (Basic Fighter Maneuvers)**: 공중전의 기본 기동 교리. 크게 세 카테고리로 나뉜다.

- **OBFM (Offensive)**: 공격 기동 — 적을 조준하고 사격 위치를 잡는 기동 (Lead/Pure/Lag Pursuit 등)
- **DBFM (Defensive)**: 방어 기동 — 적의 사격을 회피하는 기동 (Break Turn, Extension, Last Ditch 등)
- **HABFM (Head-on/Across)**: 정면 교전 기동 — 서로 마주보며 교전할 때의 기동 (1-circle, 2-circle, Scissors)

마치 격투기 선수가 상대의 움직임 패턴을 읽고(L1) 카운터 기술을 선택하고(L2) 상대의 자세에 맞춰 미세조정(L3)하는 것과 같다. 단, 상대의 "의도"를 말로 듣는 게 아니라, **관측값의 시계열 패턴**에서 직접 읽어낸다.

**현재 구현 상태 (2026-04 기준)**: L1은 98.8% 정확도로 정착, L2는 단순 CT lookup (state bin 미활용), **L3는 아직 체계화 안 됨** — SmartLeadPursuit 등 일부 노드 내부에 BFM 불변 법칙이 수작업으로 들어가 있으나 데이터 검증 경로가 없음. §11.14.8 체크리스트 참조.

**원칙 — 데이터가 패턴을 말한다 (2026-04-23 red team 교정)**:  
"어떤 상황에서 어떤 기동이 최적인가"는 **Miner/Tracker가 데이터에서 발견할 사항**. 단일 WIN match 관찰을 일반 법칙("tail chase가 정답")으로 격상 금지. Reference match는 시각적 예시로만. 이 원칙은 §11.14.3 L3, §11.15 설계 원칙, §12 Phase C 전반에 관통.

3. 관측 벡터  $o_t \in \mathbb{R}^{28}$  — 전체 정의

$\mathbb{R}^{28}$ 이란? 28개의 실수(real number)로 이루어진 벡터. 매 tick마다 시뮬레이터에서 수집되는 28개의 수치를 하나의 벡터로 묶은 것.  
매 틱(0.2초)마다 수집되는 28개 수치. 세 그룹으로 구성된다.

3.1 연속형 (Continuous) — 14개

| #  | 변수명                  | 영문 풀네임                | 물리 의미                                                     | 단위 규약      |
|----|----------------------|-----------------------|-----------------------------------------------------------|------------|
| 1  | distance_ft          | Distance              | 나 ↔ 적 거리                                                  | raw (ft)   |
| 2  | ata_deg              | Antenna Train Angle   | 적이 나를 바라보는 각도                                             | ×180 → 도   |
| 3  | aa_deg               | Aspect Angle          | 내가 적을 바라보는 각도                                             | ×180 → 도   |
| 4  | hca_deg              | Hot/Cold Angle        | 교전 기하학 각도                                                 | ×180 → 도   |
| 5  | relative_bearing_deg | Relative Bearing      | 상대 방위각                                                    | ×180 → 도   |
| 6  | ego_altitude_ft      | Ego Altitude          | 내 고도 ( ego = 나 자신을 가리키는 코드 용어)                            | raw (ft)   |
| 7  | ego_vc_kts           | Calibrated Airspeed   | 내 속도 (계기 대기속도: 고도·밀도를 보정한 실용 속도 단위)                       | raw (kts)  |
| 8  | specific_energy_ft   | Specific Energy $E_s$ | 단위 중량당 총 역학 에너지                                           | raw (ft)   |
| 9  | ps_fts               | Specific Excess Power | 비에너지 변화율 $\dot{E}_s$ (양수=에너지 얻는 중, 음수=소모 중)               | raw (ft/s) |
| 10 | energy_diff_ft       | Energy Differential   | 나 - 적 비에너지 차이 (양수=내가 우세)                                  | raw (ft)   |
| 11 | closure_rate_kts     | Closure Rate          | 접근속도 (양수=서로 가까워짐, 음수=멀어짐)                                 | raw (kts)  |
| 12 | turn_rate_degs       | Turn Rate             | 내 선회율 (초당 방향 변화각)                                         | raw (°/s)  |
| 13 | alt_gap_ft           | Altitude Gap          | 나 - 적 고도차 (양수=내가 위, 음수=적이 위)                              | raw (ft)   |
| 14 | tau_deg              | Tau                   | 시간 여유각. 현재 기동을 유지했을 때 충돌까지 걸리는 시간을 각도로 환산한 지표. 값이 작을수록 위험 | ×180 → 도   |

⚠ 단위 규약 위반 = BUG-4 수준의 silent failure.  
ata\_deg 등 \_deg 변수는 내부 저장값이 0~1 범위이며, 도(°)로 쓰려면 반드시 ×180 해야 한다.

비에너지 (Specific Energy) 상세

$$E_s = \frac{E_{\text{total}}}{W} = \frac{mgh + \frac{1}{2}mv^2}{mg} = h + \frac{v^2}{2g}, \quad g = 32.174 \text{ ft/s}^2$$

- 왜 "비(比, specific)"인가: 질량(또는 중량  $W = mg$ )으로 나눠 단위 중량당 에너지로 만든 것. 기체 무게에 상관없이 전투 여력을 비교할 수 있다.
- 단위가 ft인 이유: 질량으로 나누면 단위가 J/N = m (또는 ft)로 떨어진다. "에너지 고도(Energy Height)"라고도 부른다.
- 전술적 의미: 고도 5000ft + 300kts ≡ 고도 1000ft + 400kts ( $E_s$ 가 같으면 이론상 같은 전투 여력). 기수를 올리면 속도가 고도로, 강하하면 고도가 속도로 전환된다.
- 출처: John Boyd의 Energy-Maneuverability (EM) Theory (1960s). NATO 표준 전술 교리 및 한국 공군 교범에서 표준 용어.

파생 변수:

| 변수                     | 의미                           |
|------------------------|------------------------------|
| ps_fts ( $\dot{E}_s$ ) | 양수 = 에너지 축적 중, 음수 = 에너지 소모 중 |

| 변수               | 의미                      |
|------------------|-------------------------|
| energy_diff_ft   | 양수 = 내가 에너지 우세, 음수 = 열세 |
| energy_advantage | energy_diff > 0 이면 1    |

실험 결과: WIN 매치는 energy\_diff 평균이 낮았다 — 이기는 쪽이 에너지를 아끼지 않고 적극 소모했다는 뜻.

### 3.2 이진형 (Binary Flag) — 7개

| #  | 변수명              | 영문 풀네임                    | 의미                                                                     |
|----|------------------|---------------------------|------------------------------------------------------------------------|
| 15 | in_wez           | In Weapon Engagement Zone | 내가 WEZ 안에 있는가 (적이 나를 쏠 수 있는가)                                          |
| 16 | enm_in_wez       | Enemy In WEZ              | 적이 WEZ 안에 있는가 (내가 쏠 수 있는가)                                             |
| 17 | in_39_line       | In 3-9 Line               | 내가 적의 전방 반구(3-9 line 앞쪽) 안에 있는가                                        |
| 18 | overshoot_risk   | Overshoot Risk            | 오버슈트 위험. 내 속도가 너무 빨라 적을 추격하다 앞질러 지나칠 가능성. 오버슈트가 발생하면 공격자와 피공격자 위치가 역전됨 |
| 19 | energy_advantage | Energy Advantage          | 내 비에너지 > 적 비에너지                                                        |
| 20 | alt_advantage    | Altitude Advantage        | 내 고도 > 적 고도                                                            |
| 21 | spd_advantage    | Speed Advantage           | 내 속도 > 적 속도                                                            |

WEZ (Weapon Engagement Zone): Distance 152~914 ft + ATA < 12°. 이 조건이 동시에 만족되면 사격 유효 구간.

### 3-9 Line 상세

3-9 line은 전투기를 위에서 내려다봤을 때 **양쪽 날개 끝을 이은 가상의 선**이다. 시계 문자판 기준으로 3시(오른쪽 날개)와 9시(왼쪽 날개)를 잇는다.

 3-9 line 개념도

이 선이 항공기 주변 공간을 두 반구로 나눈다:

| 위치                    | in_39_line 값 | 전술적 의미                                   |
|-----------------------|--------------|------------------------------------------|
| 전방 반구 (12시 방향, 기수 앞쪽) | 1            | 적 기수가 나를 향함 → 적이 나를 쏠 가능성 있음 → 방어적 기동 고려 |
| 후방 반구 (6시 방향, 꼬리 뒤쪽)  | 0            | 내가 적의 등 뒤를 잡은 상황 → 공격하기 유리한 위치           |

요약: in\_39\_line = 1 이면 내가 적의 사격 위협권 안에 있다는 뜻이고, 0 이면 내가 공격 우위 위치에 있다는 뜻이다.

### 3.3 BFM One-hot — 7개

현재 내 BT가 어떤 카테고리의 기동을 실행 중인지를 **원-핫 인코딩(one-hot encoding)** 으로 표현한다.

**원-핫 인코딩이란?**  
7개 자리 중 현재 해당하는 카테고리 하나만 1로 표시하고 나머지는 전부 0으로 채우는 방식.  
예) 현재 DBFM(방어 기동) 중이라면:

| OBFM | DBFM | HABFM | UNKNOWN | UNK_NEAR_OFF | UNK_SCISSORS | UNK_DISENGAGING |
|------|------|-------|---------|--------------|--------------|-----------------|
| 0    | 1    | 0     | 0       | 0            | 0            | 0               |

이렇게 하는 이유: "기동 카테고리"는 이름(범주형 데이터)이라 신경망에 숫자로 넘겨야 한다.  
단순히 OBFM=1, DBFM=2, HABFM=3 처럼 번호를 매기면 HABFM > DBFM 같은 크기 관계가 생겨 모델이 잘못 학습하므로, 원-핫으로 표현해 카테고리 간 순서·크기 관계를 없앤다.

| #  | 변수명             | 의미                                                             |
|----|-----------------|----------------------------------------------------------------|
| 22 | OBFM            | Offensive BFM — 공격 기동 중 (Lead/Pure/Lag Pursuit, Gun)           |
| 23 | DBFM            | Defensive BFM — 방어 기동 중 (Break Turn, Extension, Last Ditch)    |
| 24 | HABFM           | Head-on/Across BFM — 정면 교전 기동 중 (1-circle, 2-circle, Scissors) |
| 25 | UNKNOWN         | 미분류                                                            |
| 26 | UNK_NEAR_OFF    | 근거리 공격 변형                                                      |
| 27 | UNK_SCISSORS    | 시저스 기동 변형                                                      |
| 28 | UNK_DISENGAGING | 이탈 기동 중                                                        |

## 4. 시스템 전체 구조 — $\pi$ 의 정의

### 4.1 정책 함수 $\pi$

**정책 함수(Policy Function)  $\pi$ 란?** 인간 조종사의 "판단"에 해당하는 함수. "현재 상황을 관찰하고 → 다음에 어떤 행동을 할지 결정하는" 역할. 강화학습에서 agent의 행동 전략 전체를  $\pi$ 로 표기하는 것이 관례다.

$$\pi : \mathbb{R}^{K \times 28} \rightarrow \mathcal{A}$$

$\pi$ 는 K=20 길이의 관측 시퀀스를 받아서 BFM 기동 하나를 리턴하는 함수다.

**K=20이 4초인 이유:** 시뮬레이터의 제어 주기는 0.2초/tick이다. 20 tick × 0.2초 = **4초치 관측**. 4초는 공중전에서 기동 의도가 명확하게 드러나기에 충분한 시간창(window)으로 설정됐다.

$$\pi(\mathbf{O}_t) = \text{CT} \left[ \arg \min_{c \in \mathcal{C}} \|\phi(\mathbf{O}_t) - p_c\|_2 \right]$$

⚠ **현재 수식은 2-레이어(L1+L2)만 포착** — 완전한 3-레이어 정책은 자기·적 관측 및 state bin을 모두 입력으로 받고 노드와 파라미터 튜플을 반환한다:

$$\pi^*(\mathbf{O}_t^{\text{ego}}, \mathbf{O}_t^{\text{enm}}, s_t) = (\text{node} = \text{CT}[\hat{c}_t, s_t], \theta = f_{\text{node}}(o_t^{\text{ego}}, o_t^{\text{enm}}))$$

여기서:

- $\mathbf{O}_t^{\text{enm}}$ : 적의 관측 시퀀스 (§11.14.4-B의 enemy observation channel이 필요)
- $s_t$ : 4D state bin (*ata, dist, closure, e\_diff*) (§11.14.2)
- $\hat{c}_t = \arg \min_c \|\phi(\mathbf{O}_t^{\text{ego}}) - p_c\|_2$ : L1 분류
- $\theta$ : 노드 내부 파라미터 벡터 (§11.14.3 L3)

현재 구현된  $\pi$ 는 위 수식의 단순화 —  $s_t, \mathbf{O}_t^{\text{enm}}, \theta$ 를 모두 생략한 2-레이어 근사.

각 기호의 정의:

| 기호                                                            | 정의                                  |
|---------------------------------------------------------------|-------------------------------------|
| $\mathbf{O}_t = [o_{t-19}, \dots, o_t]$                       | 현재 틱 기준 지난 4초치 관측 시퀀스               |
| $\phi : \mathbb{R}^{K \times 28} \rightarrow \mathbb{R}^{64}$ | GRU 인코더 — 시퀀스를 64차원 L2-정규화 벡터로 압축   |
| $p_c \in \mathbb{R}^{64}$                                     | 클래스 $c$ 의 프로토타입 벡터 (학습 데이터 임베딩의 평균) |

| 기호                 | 정의                                                        |
|--------------------|-----------------------------------------------------------|
| $\mathcal{C}$      | {CLOSING, EXTENDING, ORBITING, CLIMBING, DIVING, GUN_RUN} |
| $\text{CT}[\cdot]$ | Counter Table — 의도 클래스 → 최고 승률 기동 매핑                      |
| $\mathcal{A}$      | BFM 기동 집합 (SmartHighYoYo, SmartBreakTurn 등 23종)           |

arg min이란?

"뒤에 오는 식을 **최소**로 만드는 **입력값**을 골라라"는 뜻이다. min 과의 차이:

- min → 가장 작은 **값** 자체를 리턴 (예: 0.19)
- argmin → 가장 작은 값을 만드는 **입력** 을 리턴 (예: CLOSING)

여기서  $\|\phi(\mathbf{O}_t) - p_c\|_2$ 는 쿼리 벡터와 프로토타입  $p_c$  사이의 \*\*Euclidean 거리(직선 거리)\*\*다.  
**거리가 작다 = 더 가깝다 = 더 비슷하다**이므로, arg min으로 거리가 가장 작은 클래스를 고르는 것이 곧 **가장 유사한 클래스를 선택**하는 것과 같다.

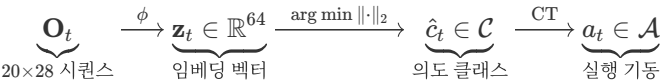
$$\arg \min_c \|\phi - p_c\|_2 = \text{”나와 가장 가까운(= 가장 닮은) 클래스를 골라라”}$$

혼동 포인트: "유사도가 높은 것을 고른다"고 하면 arg max처럼 느껴지지만, 이 수식은 유사도가 아니라 **거리**를 기준으로 쓰여 있기 때문에 arg min이 맞다. arg max를 쓰려면 식을 음의 거리나 코사인 유사도 등으로 바꿔야 한다.  
6개 클래스 각각에 대해 거리를 계산하고, **거리가 가장 짧은 클래스 이름을 리턴**한다.

| 클래스 $c$   | $\ \phi - p_c\ _2$ |
|-----------|--------------------|
| CLOSING   | 0.19 ← argmin이 선택  |
| GUN_RUN   | 0.68               |
| CLIMBING  | 0.94               |
| DIVING    | 1.05               |
| ORBITING  | 1.21               |
| EXTENDING | 1.43               |

결과:  $\hat{c}_t = \text{CLOSING}$

4.2 펼쳐서 보면



각 단계의 데이터 형태 변화:

[ O\_t ]

t-19 | 0.72 0.09 ... |

t-18 | 0.68 0.10 ... |

t-17 | 0.71 0.10 ... |

⋮ | ⋮ |

t-1 | 0.65 0.12 ... |

t | 0.63 0.13 ... |

[ z\_t ]

| 0.72 |

| -0.18 |

| 0.55 |

| ⋮ |

| 0.31 |

| ⋮ |

| 0.44 |

[ c\_t ]

CLOSING

(argmin)

[ a\_t ]

SmartHighYoYo

shape: (20, 28)

dtype: float

560개 수치

shape: (64,)

dtype: float

L2 norm = 1

enum string

"CLOSING"

6개 중 1개

action string

"SmartHighYoYo"

23개 중 1개

클래스별 거리 (argmin 계산 과정):

|           |                                                                                   |      |           |
|-----------|-----------------------------------------------------------------------------------|------|-----------|
| CLOSING   |   | 0.19 | ← 최소 → 선택 |
| GUN_RUN   |  | 0.68 |           |
| CLIMBING  |  | 0.94 |           |
| DIVING    |  | 1.05 |           |
| ORBITING  |  | 1.21 |           |
| EXTENDING |  | 1.43 |           |

## 5. EIM 내부 구조 (Trajectory ProtoNet)

### 5.1 인코더 $\phi$ 상세

입력:  $O_t$  (shape:  $20 \times 28$ )  
 ↑ 20 tick(4초) × 28개 관측값

↓

GRU

- input\_size = 28 ← 매 tick의 입력 크기 (관측 벡터 차원)
- hidden\_dim = 128 ← 내부 기억 벡터 크기. 128개 숫자가 "지금까지 본 패턴"을 요약
- num\_layers = 2 ← GRU를 2층으로 쌓음. 1층: 저수준 패턴(거리/속도 변화), 2층: 고수준 전술 패턴
- dropout = 0.1 ← 학습 시 뉴런 10%를 랜덤 비활성화 → 과적합 방지

↓

출력:  $h_1, h_2, \dots, h_{20}$  (각각 128차원)

↓

Attention Pooling

- Linear(128 → 1) + softmax → 가중합
- 20개의 128차원 벡터를 중요도 가중치로 합산 → 128차원 벡터 1개

↓

Projection Head

- Linear(128 → 128) → ReLU → Dropout(0.1) → Linear(128 → 64)
- 128차원을 64차원으로 압축. 64는 분류에 필요한 최소 정보를 담도록 실험적으로 결정된 값

↓

L2 Normalize →  $z_t \in \mathbb{R}^{64}$  ( $\|z_t\|_2 = 1$ )

↑ 64차원 벡터를 단위 길이로 정규화. 이후 거리 비교가 방향(패턴)만 반영하도록

### GRU 상세 — 왜 일반 RNN이 아닌가

#### 먼저: 일반 RNN의 문제

일반 RNN은 매 tick마다 hidden state를 단순히 덮어쓴다.

$$h_t = \tanh(W \cdot x_t + U \cdot h_{t-1})$$

tick이 20개밖에 안 되더라도, 역전파(backpropagation) 시 기울기가 tick을 거슬러 올라갈수록 점점 작아진다(vanishing gradient). 결과적으로 **t-19의 패턴이  $h_{20}$ 에 거의 남지 않게 된다**. CLOSING처럼 "20 tick 내내 지속되는 추세"를 학습하는 데 치명적이다.

#### GRU의 해결책: 두 개의 게이트

GRU는 매 tick마다 두 가지 질문을 한다.

**Reset Gate:** "이전 기억을 얼마나 버릴까?"

**Update Gate:** "새 정보를 얼마나 반영할까?"

$$r_t = \sigma(W_r \cdot x_t + U_r \cdot h_{t-1}) \quad \leftarrow \text{Reset Gate} \quad (0 \sim 1)$$

$$z_t = \sigma(W_z \cdot x_t + U_z \cdot h_{t-1}) \quad \leftarrow \text{Update Gate} \quad (0 \sim 1)$$

두 게이트 모두 sigmoid를 써서 출력이 0~1 사이이다. 0에 가까울수록 "무시", 1에 가까울수록 "반영".

각 게이트의 역할을 공중전으로 직관화하면:

| 상황                        | Reset Gate                 | Update Gate                          |
|---------------------------|----------------------------|--------------------------------------|
| 적이 갑자기 GUN_RUN으로 전환       | 높게 활성화 → 이전 ORBITING 기억 리셋 | 높게 활성화 → 새 입력(거리 급감, ATA 급감)을 강하게 반영 |
| CLOSING 추세가 20 tick 내내 지속 | 낮게 유지 → 이전 기억 보존           | 낮게 유지 → 쌓인 추세 정보를 계속 이어감             |

매 tick에서 실제로 일어나는 일:

```
# 1. Candidate hidden state (새로 제안되는 기억)
h_tilde_t = tanh(W · x_t + U · (r_t ⊙ h_{t-1}))
#                               ↑ Reset Gate가 이전 기억 중 얼마를 참고할지 결정

# 2. 최종 hidden state (과거와 현재를 혼합)
h_t = (1 - z_t) ⊙ h_{t-1} + z_t ⊙ h_tilde_t
#       ↑ 이전 기억 유지 비율   ↑ 새 정보 반영 비율

⊙ 는 원소별 곱(element-wise multiply). z_t가 0이면 h_{t-1}을 그대로 유지, z_t가 1이면 h_tilde_t로 완전히 교체.
```

20 tick 흐름 예시 (CLOSING 시나리오):

```
tick t-19: x = [dist=7000, closure=+265, ata=0.09, ...]
           h_1 = GRU(x, h_0)
           → "접근 중인 것 같다. 아직 확산 없음."

tick t-15: x = [dist=6200, closure=+272, ata=0.10, ...]
           Reset Gate 낮음 → 이전 기억 보존
           Update Gate 낮음 → 추세 누적
           h_5 = "접근 추세 지속. 점점 확산."
```

GRU를 2 layers로 쌓은 이유: 1층이 "거리/속도 변화 같은 저수준 패턴"을 잡고, 2층이 "그 패턴들의 조합으로 만들어지는 고수준 전술 패턴"을 잡는다.

Attention Pooling 상세 — 어느 tick이 중요한가

GRU가 뽑은  $h_1, h_2, \dots, h_{20}$  (각각 128차원)을 단순 평균하면 정보가 희석된다. Attention은 각 tick의 중요도 가중치를 학습한다.

```
e_k = Linear(h_k)           # 128 → 1, 각 tick의 "중요도 점수"
w_k = softmax([e_1, ..., e_20]) # 합이 1이 되도록 정규화
출력 = Σ w_k · h_k          # 가중합
```

패턴별 Attention 분포:



CLOSING (전체 추세 중요):  
w: [0.05, 0.05, 0.05, 0.05, 0.05, ..., 0.05, 0.05] ← 고르게 분포

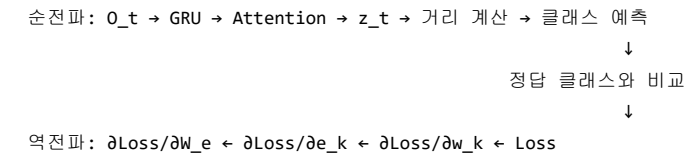
GUN\_RUN (마지막 순간이 결정적):  
w: [0.01, 0.01, ..., 0.02, 0.08, 0.25, 0.38] ← 뒤에 집중

EXTENDING (이탈 시작 순간이 핵심):  
w: [0.31, 0.22, 0.15, 0.10, ..., 0.02, 0.01] ← 앞에 집중

이 가중치는 사람이 지정하는 게 아니라, "어떻게 attention을 분배해야 분류가 잘 되는가"를 역전파로 자동 학습한다.

역전파(Backpropagation)란?

순전파(forward pass)가 "입력 → 출력을 계산하는 과정"이라면, 역전파는 "출력이 얼마나 틀렸는지를 거꾸로 흘려보내서 각 파라미터를 얼마나 수정해야 하는지 계산하는 과정"이다.



Attention에서 실제로 학습되는 파라미터는 딱 하나다:

w\_e : shape (128, 1) ← 이것만 학습됨

**w\_e가 무엇인가?** w\_e는 Attention의 가중치 행렬(Weight matrix)이다. shape (128, 1)은 "128개의 입력을 받아 숫자 1개를 출력하는 선형 변환"이라는 뜻이다. 구체적으로:

h\_k : 128차원 벡터 (GRU가 tick k에서 만든 "기억 요약")  
w\_e : (128, 1) 행렬 (128개 입력 → 1개 출력으로 압축하는 학습 파라미터)

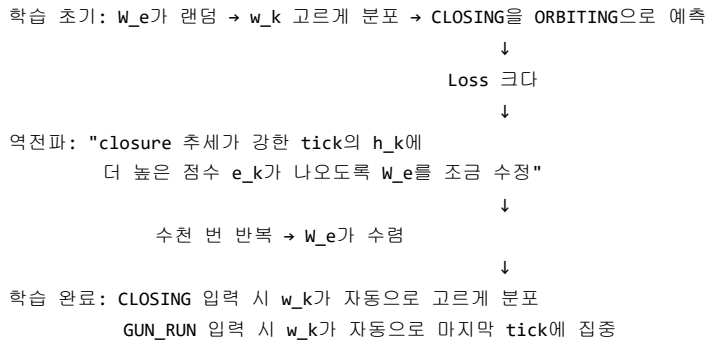
$e_k = h_k \cdot w_e$  (내적 연산: 128차원 → 스칼라 1개)  
→ "이 tick이 얼마나 중요한가?"를 나타내는 점수

w\_e의 각 원소는 "h\_k의 어떤 차원(특징)이 중요도 판단에 얼마나 기여하는가"를 나타낸다. 역전파를 통해 "closure 추세를 잘 담은 차원에 높은 가중치"를 주는 방향으로 수렴한다.

신경망 기본 연산 용어 정리:

| 용어           | 의미                                                    | 이 시스템에서의 역할                           |
|--------------|-------------------------------------------------------|---------------------------------------|
| Linear(a→b)  | 선형 변환. a차원 입력을 b차원 출력으로 변환하는 행렬 곱                     | 차원 압축/변환 (128→64 등)                   |
| softmax      | 여러 숫자를 "합이 1인 확률 분포"로 변환. 가장 큰 값이 가장 높은 확률을 가짐        | Attention 가중치 w_k를 확률로 정규화            |
| sigmoid σ(x) | 임의의 숫자를 0~1 사이 값으로 압축. $\sigma(x) = 1 / (1 + e^{-x})$ | GRU의 Reset/Update Gate가 0~1 사이 값을 내도록 |
| tanh(x)      | 임의의 숫자를 -1~1 사이 값으로 압축                                | GRU의 Candidate hidden state 계산        |
| ReLU(x)      | max(0, x). 음수를 0으로 만들고 양수는 그대로 통과                     | Projection Head의 비선형성 추가              |
| Dropout(p)   | 학습 시 p% 뉴런을 랜덤 비활성화 → 과적합 방지                          | Dropout(0.1) = 10% 비활성화               |

### 역전파로 W\_e가 학습되는 과정 (CLOSING 예시):



핵심: W\_e 하나가 역전파 신호를 통해 "어느 tick에 집중해야 Loss가 줄었는가"를 패턴별로 알아서 학습한다. 분류 성능이 올라갈수록 Loss가 줄고, Loss가 줄수록 W\_e가 더 정확하게 수렴한다.

## Projection + L2 정규화 — 64차원으로 압축 후 정규화

Attention 출력: 128차원

- Linear(128→128) → ReLU → Dropout(0.1)
- Linear(128→64)
- L2 Normalize:  $z_t = z / \|z\|_2$  ( $\|z_t\|_2 = 1$ )

L2 정규화를 하는 이유: ProtoNet의 거리 계산( $\|z_t - p_c\|_2$ )이 벡터의 크기(magnitude)가 아닌 **방향(direction)** 만으로 비교되어야 하기 때문이다. 정규화 후엔 모든 벡터가 64차원 단위구(unit sphere) 위에 놓여서, 거리가 순수하게 패턴의 유사성만 반영한다.

## 5.2 프로토타입 분류 (Prototypical Network)

$$\hat{c}_t = \arg \min_{c \in \mathcal{C}} \|z_t - p_c\|_2$$

### 직관: "평균 얼굴"과의 거리

각 클래스의 prototype  $p_c$ 는 그 클래스 학습 샘플들의 임베딩 평균이다. 마치 수백 명의 "CLOSING 패턴" 얼굴을 합성한 "평균 CLOSING 얼굴"을 만들어두는 것과 같다. 새 입력이 들어오면 6개 평균 얼굴 중 누구와 가장 닮았는지 거리로 판단한다.

### 학습 시 — prototype 만들기:

CLOSING 학습 샘플 수백 개

- 각각  $\phi(\text{GRU+Attn+Proj})$  통과
- 64차원 벡터 수백 개 (모두 단위구 위)
- 평균 →  $p_{\text{CLOSING}} = [0.81, -0.12, 0.44, \dots]$

EXTENDING 학습 샘플 수백 개

- 각각  $\phi$  통과
- 평균 →  $p_{\text{EXTENDING}} = [-0.73, 0.55, -0.31, \dots]$

... 6개 클래스 동일하게 반복

### 추론 시 — 거리로 분류:

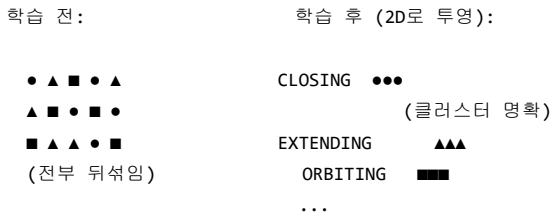
새  $0_t \rightarrow \phi \rightarrow z_t = [0.72, -0.18, 0.55, \dots]$

$\|z_t - p_{\text{CLOSING}}\| = 0.19 \leftarrow$  가장 가까움  $\rightarrow$  선택  
 $\|z_t - p_{\text{GUN\_RUN}}\| = 0.68$   
 $\|z_t - p_{\text{CLIMBING}}\| = 0.94$   
 $\|z_t - p_{\text{DIVING}}\| = 1.05$   
 $\|z_t - p_{\text{ORBITING}}\| = 1.21$   
 $\|z_t - p_{\text{EXTENDING}}\| = 1.43$

$\rightarrow \hat{c}_t = \text{CLOSING}$

왜 학습이 제대로 되냐면 — 임베딩 공간을 조각내는 훈련:

$\phi$ (인코더 전체)는 "같은 클래스끼리는 가깝게, 다른 클래스끼리는 멀게" 배치되도록 훈련된다. 처음에는 6개 클래스가 뒤섞여 있지만, 학습이 진행되면서 64차원 공간에 6개의 뚜렷한 클러스터가 형성된다.



Trajectory 라벨(B안)이 98.8%가 나오는 이유: closure\_rate > +100 지속 이라는 물리 패턴은 상대가 누구든 GRU가 항상 비슷한 방향으로 인코딩하기 때문에 클러스터가 깔끔하게 분리된다. 반면 Node-based 라벨(A안)은 같은 노드 이름이라도 파라미터에 따라 전혀 다른 궤적을 만들어서 클러스터가 겹친다.

5.3 Trajectory 라벨 정의 (학습 데이터 라벨링 규칙)

```
def trajectory_label(window):
    closure_mean = mean(window["closure_rate_kts"])
    dist_mean    = mean(window["distance_ft"])
    ata_mean     = mean(window["ata_deg"]) * 180 # 단위 변환
    alt_delta    = window["alt_gap_ft"][-1] - window["alt_gap_ft"][0]

    if dist_mean < 1500 and ata_mean < 20: return "GUN_RUN"
    if closure_mean > +100:                return "CLOSING"
    if closure_mean < -100:                return "EXTENDING"
    if alt_delta > +500:                   return "CLIMBING"
    if alt_delta < -500:                   return "DIVING"
    return "ORBITING"
```

| 클래스       | 판정 기준                     | BFM 해석    | 대응 기동          |
|-----------|---------------------------|-----------|----------------|
| GUN_RUN   | dist < 1500ft + ATA < 20° | 사격 시도 중   | SmartBreakTurn |
| CLOSING   | closure_mean > +100 kts   | 빠르게 접근 중  | SmartHighYoYo  |
| EXTENDING | closure_mean < -100 kts   | 이탈 중      | SmartLowYoYo   |
| CLIMBING  | 고도 +500ft 이상 상승/window    | 에너지 축적 중  | SmartLowYoYo   |
| DIVING    | 고도 -500ft 이상 하강/window    | 에너지→속도 전환 | SmartHighYoYo  |
| ORBITING  | 나머지                       | 교착·선회 반복  | LeadPursuit    |

5.4 왜 Node-based(A안, 73.7%)보다 Trajectory-based(B안, 98.8%)가 훨씬 정확한가

- A안: 적의 active\_node 이름을 라벨로 사용. 같은 노드 이름이라도 파라미터에 따라 완전히 다른 궤적 → 클러스터가 섞임.

- **B안:** 관측 시퀀스 자체의 물리 패턴을 라벨로 사용. 노드 이름을 모르는 미지 상태에도 동작. 물리적으로 같은 패턴은 임베딩 공간에서 같은 클러스터를 형성 → 깔끔하게 분리됨.

⚠ **98.8%에 대한 red team 캐비어트** — 이 수치 자체로는 불충분한 근거다. §11.15.3의 세 가지 검증이 모두 필요:

1. **Per-class accuracy** — 통합 98.8%가 ORBITING 클래스 편중일 가능성. ORBITING은 "나머지" 조건이라 학습 데이터 비율 지배적 → "전부 ORBITING으로 찍어도 accuracy 높음" 구조. 각 클래스별 precision/recall 별도 보고 필요.
2. **Calibration** —  $\text{conf} \geq 0.50$  게이트가 의미 있으려면 confidence가 실제 정확도와 정합해야 함. Reliability diagram 미확인 →  $\text{conf}=0.6$  이 실제로 60% 정확도를 의미하는지 모름. BT의 EnemyIntentIs Condition (§11.8)의 게이트가 허상일 수 있음.
3. **Label quality audit** — 98.8%는 *rule-based trajectory\_label()*에 대한 정확도다. Rule 자체가 옳다는 보장은 없음. 예: CLOSING+CLIMBING 복합 상황은 rule에서 CLOSING으로 잘림 (§5.5) → EIM이 "rule을 정확히 학습"해도 전술적으로 틀릴 수 있음.

695 폴에서 adaptive BT가 static 대비 +0.05pp밖에 못 얻는 현상 (§1 표)이 이 세 가지 결함의 종합 증상일 가능성.

## 5.5 클래스 설계 분석 및 한계 — 6개는 충분한가?

### 현재 6개 클래스의 근거

6개 클래스는 물리 관측값 3가지만으로 판정한다.

```
closure_rate → CLOSING / EXTENDING
alt_delta    → CLIMBING / DIVING
dist + ATA   → GUN_RUN
나머지       → ORBITING
```

단순하고 해석 가능하며, 6개 상태 폴에서는 EIM 정확도 98.8%가 나왔다.

### 충분하지 않은 근거

#### 문제 1: ORBITING이 "쓰레기통 클래스"다

ORBITING은 위 5개 조건에 해당하지 않는 모든 상황을 묶는다. 실제로는 전혀 다른 전술들이 뒤섞인다.

ORBITING으로 분류되는 실제 상황들:

- 1-circle 선회전: 낮은 속도, 좁은 선회 반경 → SmartOneCircle이 최적
- 2-circle 선회전: 높은 속도, 넓은 선회 반경 → SmartTwoCircle이 최적
- Scissors: 적의 오버슈트 유도 중 → FlatScissors가 최적
- 교착 상태: 서로 비슷한 위치에서 맴돌 → LeadPursuit (현재 대응)

1-circle과 2-circle의 최적 대응이 다름에도 불구하고 모두 LeadPursuit으로 처리된다.

#### 문제 2: 복합 패턴이 우선순위 규칙으로 잘림

```
if closure_mean > +100: return "CLOSING" # ← 먼저 체크됨
if alt_delta > +500: return "CLIMBING" # ← 이미 CLOSING이면 도달 못 함
```

"접근하면서 동시에 상승(High YoYo 기동)"하는 상황은 CLOSING으로만 분류된다. CLIMBING 신호가 무시되어 SmartLowYoYo 대신 SmartHighYoYo가 선택되는데, 이 두 대응은 전혀 다른 기동이다.

#### 문제 3: 695 폴 WR 55%의 원인과 직결

Coverage gap 26.9%의 상당 부분이 "6개 클래스 경계 사이 애매한 구간"일 가능성이 높다. 예컨대 closure\_rate 가 +80~+100 kts인 구간은 CLOSING도 ORBITING도 아닌 모호한 경계 영역이다.

추가할 수 있는 클래스 후보

| 후보 클래스    | 판정 기준                                   | 현재 분류     | 필요한 이유                         |
|-----------|-----------------------------------------|-----------|--------------------------------|
| 1-CIRCLE  | dist < 4000 + turn_rate 높음 + closure 낮음 | ORBITING  | 1-circle vs 2-circle 대응 기동이 다름 |
| 2-CIRCLE  | dist > 4000 + speed 높음 + closure 낮음     | ORBITING  | SmartTwoCircle이 최적             |
| SCISSORS  | overshoot_risk=1 + closure 급변           | ORBITING  | 적이 오버슈트를 의도적으로 유도 중            |
| HIGH_YOYO | closure > +100 + alt_delta > +300 동시    | CLOSING   | 접근 + 상승이 결합된 에너지 기동            |
| DEFENSIVE | DBFM=1 + closure < 0                    | EXTENDING | 도망이 아니라 방어 기동 중인 경우            |

Trade-off — 클래스를 늘리면 생기는 문제

클래스 수를 늘리면 두 가지 비용이 발생한다.

- 클래스 수 ↑ → 클래스당 학습 샘플 수 ↓
  - 각 prototype의 대표성 약화
  - 클래스 경계 중첩 구간 증가
  - EIM 정확도 하락 가능성

즉 무조건 세분화가 좋은 게 아니다. 클래스를 추가할 때는 반드시 충분한 학습 샘플 확보가 전제되어야 한다.

현실적 개선 방향

ORBITING만 분할하는 것이 가장 효과 대비 리스크가 낮다.

- 현재: ORBITING (1개)
- 개선: 1-CIRCLE / 2-CIRCLE / SCISSORS / NEUTRAL (4개)
- 결과: 전체 클래스 6개 → 9개

판정 기준:

```
# ORBITING 진입 후 세분화
if turn_rate > 15 and distance < 4000: return "1-CIRCLE"
if turn_rate > 10 and distance >= 4000: return "2-CIRCLE"
if overshoot_risk == 1: return "SCISSORS"
return "NEUTRAL"
```

이 개선은 Exp 2의 Coverage Gap 확장(L7 상대 생성)과 병행할 수 있는 연구 과제이며, 695 풀 WR 55% → 60%+ 목표 달성에 직접 기여할 가능성이 높다.

6. Counter Table (CT)

$$CT[c] = \arg \max_{a \in \mathcal{A}} WR(c, a)$$

수천 번의 매치 데이터에서 "적 의도가 c일 때 기동 a를 썼을 때의 승률"을 집계해서 만든 룩업 테이블.

| 의도 클래스    | 대응 기동         | BFM 논리                     |
|-----------|---------------|----------------------------|
| CLOSING   | SmartHighYoYo | 적이 돌진 중 → 상승해서 오버슈트 유도     |
| EXTENDING | SmartLowYoYo  | 적이 도망 중 → 강하 가속으로 추격       |
| ORBITING  | LeadPursuit   | 교착 → Lead angle 유지로 포인팅    |
| CLIMBING  | SmartLowYoYo  | 적이 올라가면 같이 따라 올라가서 에너지 동등화 |

| 의도 클래스  | 대응 기동          | BFM 논리                            |
|---------|----------------|-----------------------------------|
| DIVING  | SmartHighYoYo  | 적이 강하해서 속도 얻는 중 → 상승으로 맞대응        |
| GUN_RUN | SmartBreakTurn | 사격 위치 진입 → 즉각 Break Turn으로 ATA 벌림 |

⚠ **현재 CT는 single-dimension — state 무시** — 같은 CLOSING이라도 거리 1000ft vs 8000ft에서 최적 기동이 다르지만 모두 SmartHighYoYo로 일괄 처리된다. §11.14.2에서 state-bin 확장 스키마 정의:

$$CT^{(2)}[c, s] = \arg \max_a WR(c, s, a), \quad s = \text{bin}(ata, dist, closure, e\_diff)$$

런타임 TacticalLookup 노드가 이미 4D state bin을 쓰지만, 학습 파이프라인(Stage ④)이 flat CT만 만들어 두 구조가 불일치. **legacy matches.jsonl(126개)과 CSV(2400개) 재분석 시 state-bin CT 빌드 먼저.**

또한 wr 집계는 **Wilson CI 하한 정렬** 필수 (§11.15.1.5). "WR 100%(N=2)" 보다 "WR 80%(N=100)"이 상위.

**ORBITING trash-bin 주의** — ORBITING → LeadPursuit은 1-circle/2-circle/scissors/neutral을 모두 같은 기동으로 처리. §5.5의 한계가 CT 레벨에서도 그대로 전이됨. 현재 695폴 WR 55% 정체의 주 원인 후보.

7. 학습 루프 — EXPLORE → LEARN → APPLY (수식 요약)

EXPLORE:  $\mathcal{D} \leftarrow \text{simulate}(\pi, \text{opponents})$

LEARN:  $\phi, \{p_c\} \leftarrow \text{train}(\mathcal{D}), \quad CT \leftarrow \text{build}(\mathcal{D})$

APPLY:  $\pi \leftarrow \lambda \mathbf{O}_t. CT \left[ \arg \min_c \|\phi(\mathbf{O}_t) - p_c\|_2 \right]$

이 3줄은 최소 요약이고 실제 파이프라인은 9 Stage — 이 요약이 실무와 일치하려면 다음 확장이 필요:

| 수식                                       | 실제 구현 (§11.N)              | Gap                                                  |
|------------------------------------------|----------------------------|------------------------------------------------------|
| $\text{simulate}(\pi, \text{opponents})$ | Stage ① COLLECT + ② INGEST | result JSON에 seed/yaml_hash/damage_events 필요 (§11.2) |
| $\text{train}(\mathcal{D})$              | Stage ③ LEARN-EIM          | per-class accuracy + calibration 미검증 (§5.4 red team) |
| $\text{build}(\mathcal{D})$              | Stage ④ LEARN-CT           | <b>state bin 차원 빠짐</b> (§6 red team)                 |
| APPLY $\lambda$                          | Stage ⑦ APPLY              | L3 파라미터 최적화 빠짐 (§11.14.3)                            |

가설 miner와 tracker(Stage ⑤/⑥)는 이 수식에 아예 없음 — 실질적으론 LEARN 안에 끼워 넣어 생각.

전체 파이프라인은 §11.1 도식 + §11.11 한 페이지 스펙 참조.

단계별 산출물

| Stage   | 입력                | 도구                    | 산출물                                                             |
|---------|-------------------|-----------------------|-----------------------------------------------------------------|
| EXPLORE | BT YAML + 상대 폴    | collect_phase1.py     | logs/metadata/*.csv (per-tick 30 col: 관측 28개 + 매치 ID + tick 번호) |
|         | metadata CSV      | hypothesis_miner.py   | hypothesis_queue.json                                           |
|         | 가설 후보             | hypothesis_tracker.py | hypotheses.jsonl (verdict)                                      |
| LEARN   | metadata CSV      | train_intent_model.py | intent_model_trajectory.pt                                      |
|         | metadata CSV      | counter_table builder | counter_table.json                                              |
| APPLY   | intent model + CT | BT YAML 생성            | adaptive_eagle_v9.yaml                                          |

| Stage | 입력      | 도구          | 산출물                                                         |
|-------|---------|-------------|-------------------------------------------------------------|
|       | BT + 상대 | evaluate.py | WR + Wilson CI (승률 신뢰구간: 매치 수가 적을 때 승률의 오차범위를 계산) + replays |

## 8. 구체적 수치 예시 — "적이 빠르게 접근 중"

### 8.1 상황 설정

- 적기가 정면에서 속도 280kts로 돌진 중
- 거리 7000ft → 5200ft로 줄어드는 추세 (지난 4초)
- 내 고도 15,000ft, 적 고도 15,100ft (거의 동고도)

### 8.2 관측 시퀀스 (일부 컬럼)

| tick | distance_ft | closure_rate_kts | ata_deg (raw) | alt_gap_ft | enm_in_wez | OBFM |
|------|-------------|------------------|---------------|------------|------------|------|
| t-19 | 7000        | +265             | 0.09          | +180       | 0          | 0    |
| t-15 | 6200        | +272             | 0.10          | +160       | 0          | 0    |
| t-10 | 5800        | +278             | 0.11          | +140       | 0          | 1    |
| t-5  | 5500        | +281             | 0.12          | +120       | 0          | 1    |
| t    | 5200        | +285             | 0.13          | +100       | 0          | 1    |

### 8.3 Step 1 — Trajectory 라벨 계산

$$\text{closure} = \frac{1}{20} \sum_{k=0}^{19} \text{closure}_k \approx +275 \text{ kts}$$
$$+275 > +100 \Rightarrow \text{라벨} = \text{CLOSING}$$

### 8.4 Step 2 — 인코더 $\phi$ 통과

$$\mathbf{z}_t = \phi(\mathbf{O}_t) = [0.72, -0.18, 0.55, 0.31, \dots] \in \mathbb{R}^{64}, \quad \|\mathbf{z}_t\|_2 = 1$$

64개의 실수로 구성된 벡터. 각 차원이 특정 물리량에 1:1 대응되지는 않으며, GRU+Attention+Projection이 역전파를 통해 "CLOSING 패턴을 CLOSING 방향으로 밀어내는" 추상적 표현 공간을 스스로 학습한 결과다. L2 norm=1이므로 모든 벡터는 64차원 단위구(unit sphere) 위에 위치한다.

(GRU가 "접근속도 지속 증가 + 거리 감소" 패턴을 학습해서 CLOSING 방향 벡터로 압축)

### 8.5 Step 3 — 프로토타입 거리 계산

$$\|\mathbf{z}_t - p_{\text{CLOSING}}\|_2 = \mathbf{0.19} \quad \leftarrow \text{최소}$$

| 클래스      | $\ \mathbf{z}_t - p_c\ _2$ |
|----------|----------------------------|
| CLOSING  | 0.19 ★                     |
| GUN_RUN  | 0.68                       |
| CLIMBING | 0.94                       |
| DIVING   | 1.05                       |
| ORBITING | 1.21                       |

| 클래스       | $ \mathbf{z}_t - p_c _2$ |
|-----------|--------------------------|
| EXTENDING | 1.43                     |

$\hat{c}_t = \arg \min = \text{CLOSING}$

8.6 Step 4 — Counter Table 조회 (L2)

$a_t = \text{CT}[\text{CLOSING}] = \text{SmartHighYoYo}$

**BFM 논리:** 적이 빠르게 돌진 중이므로, 상승해서 에너지를 위치에너지로 전환하고 적의 오버슈트를 유도. 적이 지나치는 순간 역전해서 공격 위치 획득.

8.7 Step 5 — State bin 정제 (L2 확장, §11.14.2)

현재 CT로는 거리 5200ft든 1500ft든 동일하게 SmartHighYoYo. 실제로는 state bin 세분화 필요:

```
s_t = bin(ata=13°, dist=5200ft, closure=+285, e_diff=+100)
      = (ata_bin=0, dist_bin=2, closure_bin=3, e_diff_bin=1)

CT²[CLOSING, s_t] → "SmartHighYoYo", wr=0.74 (n=28, Wilson CI: 0.58)
alternatives: {"SmartLeadPursuit": wr=0.62 (n=19), ...}
```

5200ft는 중거리 CLOSING → SmartHighYoYo 적합. 1500ft 근접 CLOSING이었다면 SmartBreakTurn이 더 높은 WR일 수 있음.

8.8 Step 6 — L3 파라미터 최적화 (§11.14.3, §10.1 공식 적용)

선택된 SmartHighYoYo의 내부 파라미터  $\theta$ 를 §10.1 BFM Guidance Law 공식으로 계산:

[관측값으로부터 §10.1 파생값 계산]

```
o_t^ego = {ego_vc_kts=380, alt=15000, e_diff=+100, ...}
ATA = 0.13 × 180 = 23.4°, AA = 0.08 × 180 = 14.4°
dist = 5200 ft, V_c = +285 kts = 481 ft/s
V_ego = 380 kts = 641 ft/s
```

- 1. LOS 회전을 (§10.1.2):  
 $\lambda_{\dot{}} \approx \Delta \text{ATA} / \Delta t = (0.13 - 0.12) \times 180 / 0.2 = 9.0 \text{ }^\circ/\text{s}$   
→  $\lambda_{\dot{}} > 0 \rightarrow \text{Lead Pursuit}$  상태
- 2. 오버슈트 예측 (§10.1.6):  
 $t_{\text{impact}} = 5200 / 481 = 10.8\text{초}$   
 $t_{\text{turn}} = 23.4 / 15 = 1.6\text{초}$   
→  $t_{\text{impact}} \gg t_{\text{turn}} \rightarrow \text{오버슈트 위험 낮음} \rightarrow \text{적극적 Lead 허용}$
- 3. PN 파라미터 결정:  
 $\text{dist} > 3000\text{ft} + \text{에너지 우세}(+100\text{ft}) \rightarrow N = 4.0$  (적극적)  
 $K_p = 1.0, K_i = 0.1$
- 4. PN heading command (§10.1.1):  
 $a_n = N \times V_c \times \lambda_{\dot{}} = 4.0 \times 481 \times (9.0 \times \pi / 180) = 302 \text{ ft/s}^2$   
→ 강한 Lead turn 명령

$\theta = \{N: 4.0, K_p: 1.0, K_i: 0.1, \text{pitch\_gain}: 1.3, \text{vel\_cmd}: 2\}$

⚠ **현재 이 단계는 구현 안 됨** — 위 계산은 §10.1 공식의 적용 예시. SmartLeadPursuit 같은 일부 노드에서만 비슷한 로직이 hardcoded. §11.14.4의 param\_table 인프라 + §10.1 파생값 로깅 필요. **최적  $N$  값은 Miner 9가 데이터에서 발견할 대상이다** (§10.1 원칙 box).



8.9 Step 7 — 최종 action 출력

```
action = SmartHighYoYo.execute(0)
→ [alt=3, hdg=4, vel=2] # alt=climb, hdg=straight, vel=corner
```

이 [alt, hdg, vel] 3-tuple이 다음 tick의 기체 제어 입력. 한 틱 뒤 새 관측으로 Step 1부터 반복.

9. 버전 진화 요약

| 버전   | 핵심 변경                        | 6 opp WR | 695 풀 WR | 가설 ID | A/B 검증  | Reward 검증 |
|------|------------------------------|----------|----------|-------|---------|-----------|
| v6h2 | Static BT (최적 단일 전략)         | 83.3%    | 54.96%   | —     | —       | ✗ 미정의     |
| v7   | + Node-based EIM (73.7% acc) | 100%     | —        | —     | ✗ 6opp만 | ✗ 미정의     |
| v8   | StaleChaseBreak 거리 분할        | 22% ✗    | —        | 미기록   | ✗       | ✗         |
| v9   | + Trajectory EIM (98.8% acc) | 100%     | 55.01%   | 미기록   | 부분      | ✗         |

⚠ 버전 관리의 red team 지적:

- v8의 22% 폭락은 기록된 가설 ID가 없음 → 무엇을 왜 시도했고 실패 원인이 무엇인지 hypotheses.json1 에서 추적 불가. §11.15.2 원칙 6 위반.
- Reward 정의가 한 버전도 검증 안 됨 — 모든 개선 주장의 objective가 불분명 (§11.14.7).
- 695풀 검증이 v6h2와 v9에만 있음 — v7, v8은 전체 검증 없이 6-opp만 보고 판단. 과적합 감지 불가.
- "**+16.7pp on 6 opp, +0.05pp on 695**" 는 adaptive 이득이 대부분 6-opp 과적합임을 강하게 시사. Jensen 이론적 우위가 실제로 거의 실현 안 됨 (§1 caveat).

695 풀 WR이 v6h2와 거의 동일한 이유: EIM이 커버하는 관측 공간이 26.9%에 불과 → Exp 2 (Coverage Gap 확장)에서 40%+ 목표.

앞으로 새 버전 기록 시 필수 컬럼 (§11.15.2):

- 가설 ID (hypotheses.json1 링크)
- A/B baseline 명시 (which version 대비)
- 695풀 Wilson CI
- Reward 정의 링크 (§11.14.7 확정 후)
- YAML hash + SDK/EIM/CT 버전 tuple

6 opponents vs 695 풀의 차이

- 6 opponents:** 초기 개발 단계에서 사용한 소규모 검증 세트. 다양한 전술 스타일을 대표하는 6가지 상대로 빠른 가설 검증에 사용.
- 695 풀:** L1~L6 레이어로 구성된 직교 설계(Orthogonal Design) 기반 대규모 상대 풀. 관측 공간을 최대한 넓게 커버하도록 설계된 695개 상대. 실전 일반화 성능 측정용.
- 6 opponents에서 100%가 나와도 695 풀에서 55%에 그치는 이유: 6개는 EIM이 이미 학습한 패턴 범위 안에 있지만, 695개 중 상당수는 EIM이 본 적 없는 관측 공간 영역을 사용하기 때문.

**26.9% coverage의 의미:** EIM이 "자신 있게 분류할 수 있는" 관측 공간(28차원 관측값의 조합 범위)이 전체의 26.9%에 불과하다는 뜻. 나머지 73.1%의 상황에서는 EIM이 부정확한 의도 분류를 내릴 수 있어 Counter Table 선택이 틀릴 가능성이 높다.

10. 핵심 수식 모음

| 수식                 | 설명                                                                    |
|--------------------|-----------------------------------------------------------------------|
| $E_s = h + v^2/2g$ | Specific Energy (비에너지). $h$ =고도(ft), $v$ =속도(ft/s), $g$ =32.174 ft/s² |

| 수식                                                                                                                                                          | 설명                                                                                                            |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| $r \approx v^2 / (g \sqrt{n^2 - 1})$                                                                                                                        | Turn Radius (선회 반경). $n$ =load factor(중력 대비 기체에 걸리는 하중 배수, 예: $n = 5$ 이면 5G 기동). $n$ 이 클수록 선회 반경이 작아짐         |
| $\pi(\mathbf{O}_t) = \text{CT}[\arg \min_c  \phi(\mathbf{O}_t) - p_c _2]$                                                                                   | 정책 함수 (현재 구현, 2-레이어). §4의 box 참조                                                                              |
| $\pi^*(\mathbf{O}_t^{\text{ego}}, \mathbf{O}_t^{\text{enm}}, s_t) = (\text{CT}^{(2)}[\hat{c}_t, s_t], f_{\text{node}}(o_t^{\text{ego}}, o_t^{\text{enm}}))$ | 완전한 3-레이어 정책 (§11.14). L1/L2/L3 통합                                                                            |
| $\text{CT}[c] = \arg \max_a \text{WR}(c, a)$                                                                                                                | Counter Table v1 (flat, 현행)                                                                                   |
| $\text{CT}^{(2)}[c, s] = \arg \max_a \text{WR}(c, s, a)$                                                                                                    | Counter Table v2 (state-bin, §11.14.2)                                                                        |
| $r_t = ?$                                                                                                                                                   | ⚠ <b>Per-tick reward 정의 미확정</b> — <code>src/match/result.pyd</code> 확인 후 삽입 필요 (§11.14.7 action item)         |
| $\text{score} = \sum_o (W_{\text{base}} + \alpha \cdot \Delta \text{hp})$                                                                                   | Fitness Score (매치 결과 집계). $W = 10, D = 1, L = -5, \alpha = 2.0$                                               |
| $\mathbb{E}_o[\max_x f(x, o)] \geq \max_x \mathbb{E}_o[f(x, o)]$                                                                                            | Jensen 부등식 — intent 식별 정확도 × 커버리지가 충분할 때만 실현 (§1 caveat)                                                      |
| $\text{Wilson}(W, n, z = 1.96)$                                                                                                                             | 승률 신뢰구간 하한. CT best_node 선택·버전 비교 모두 이 하한으로 정렬 (§11.15.1.5)                                                   |
| Cohen's $d = (\mu_1 - \mu_2) / \sigma_{\text{pool}}$                                                                                                        | Miner 2/8의 효과 크기. 단독 사용 금지 — per-opponent stratification + BH 보정 필수 (§11.15.1)                                |
| $a_n = N \cdot V_c \cdot \dot{\lambda}$                                                                                                                     | <b>Proportional Navigation</b> (§10.1.1). $N$ =비례상수(3~5), $V_c$ =접근속도, $\dot{\lambda}$ =LOS 회전율. L3의 핵심 유도 법칙 |
| $\dot{\lambda} \approx (V_e \sin \text{AA} - V_f \sin \text{ATA}) / \text{dist}$                                                                            | <b>LOS 회전율</b> (§10.1.2). $\dot{\lambda} = 0$ 이면 Collision Course (WEZ 진입 조건)                                 |
| $\text{HCA} = \text{ATA} + \text{AA}$                                                                                                                       | <b>Intercept Geometry</b> 항등식 (§10.1.3). 세 각도의 기본 관계                                                          |
| $t_{\text{impact}} < t_{\text{turn}} \Rightarrow \text{OVERSHOOT}$                                                                                          | <b>오버슈트 예측</b> (§10.1.6). $t_i = \text{dist} / V_c, t_t = \text{ATA} / \omega_{\text{max}}$                   |

10.1 BFM Guidance Law — L3 파라미터 최적화의 수학적 기초 (2026-04-24 추가)

**배경:** L3는 "선택된 기동의 파라미터  $\theta$ 를 관측값으로 실시간 튜닝"하는 레이더다(§2, §11.14.3). 이를 구현하려면 **관측값** → **조종 입력**을 연결하는 수학적 도구가 필요하다. 전투기 기동학(BFM)과 유도 법칙(Guidance Law)에서 이미 검증된 공식들이 있으며, 이들은 **우리 관측 벡터  $o_t$ 의 변수만으로 계산 가능**하다.

**원칙 — "공식은 도구이지 규범이 아니다":** 아래 공식들은 L3의  $\theta$  후보 공간을 설계하기 위한 수학적 기초(basis function)다. "이 공식을 따르면 이긴다"라는 prescriptive rule이 **아니다**. 어떤 상황에서 어떤  $\theta$ 가 최적인지는 여전히 **Miner 9 "Param Efficacy"가 데이터에서 발견**(§11.14.3 원칙). 이 공식들은 탐색 공간을 무한→유한으로 축소하는 역할을 한다.

10.1.1 Proportional Navigation (PN) — 핵심 유도 법칙

미사일 유도에서 유래했으나, **Lead Pursuit의 수학적 기초**이기도 하다.

$$a_n = N \cdot V_c \cdot \dot{\lambda}$$

| 기호              | 정의                                  | 우리 관측 벡터 매핑                                    | 단위                |
|-----------------|-------------------------------------|------------------------------------------------|-------------------|
| $a_n$           | LOS 수직 방향 가속도 (조종 입력)               | → <b>heading command 변화량</b>                   | ft/s <sup>2</sup> |
| $N$             | Navigation Constant (비례 상수, 통상 3~5) | → <b>L3 파라미터 <math>\theta_1</math></b> (튜닝 대상) | 무차원               |
| $V_c$           | 접근 속도 (Closing Velocity)            | <code>closure_rate_kts</code> × 1.6878         | ft/s              |
| $\dot{\lambda}$ | LOS 회전율 (Line-of-Sight rate)        | 아래 §10.1.2에서 계산                                | rad/s             |

**해석:** "시선선(LOS)이 회전하는 속도에 비례하여 기수를 틀어라".  $N$ 이 크면 더 적극적으로 Lead Pursuit,  $N$ 이 작으면 Lag Pursuit에 가까워진다.

$N \uparrow \Rightarrow \text{Lead Pursuit (적극적 추적)}, \quad N \downarrow \Rightarrow \text{Lag Pursuit (보수적 추적)}$

**L3 적용:**  $N$ 을 고정값이 아닌 **관측값의 함수**  $N(o_t) = f(\text{dist}, \text{closure}, \text{ATA})$ 로 만들면, 같은 LeadPursuit 노드라도 상황에 따라 다른 추적 강도를 보인다. 이것이 L3의 핵심 메커니즘이다.

### 10.1.2 LOS 회전을 $\dot{\lambda}$ — 관측 벡터에서 직접 계산

$$\dot{\lambda} \approx \frac{V_{\text{enm}} \cdot \sin(\text{AA}) - V_{\text{ego}} \cdot \sin(\text{ATA})}{\text{dist}}$$

| 변수               | 관측 벡터 필드                                  | 단위 변환                  |
|------------------|-------------------------------------------|------------------------|
| $V_{\text{ego}}$ | ego_vc_kts $\times$ 1.6878                | kts $\rightarrow$ ft/s |
| $V_{\text{enm}}$ | 직접 관측 없음 — <b>적 속도 추정 필요</b> (§11.14.4-B) | ft/s                   |
| ATA              | ata_deg $\times$ 180                      | 내부값 $\rightarrow$ 도    |
| AA               | aa_deg $\times$ 180                       | 내부값 $\rightarrow$ 도    |
| dist             | distance_ft                               | raw ft                 |

세 가지 물리 상태:

| $\dot{\lambda}$ 부호  | 의미             | Pursuit 유형              | 전술적 상황           |
|---------------------|----------------|-------------------------|------------------|
| $\dot{\lambda} > 0$ | LOS가 전방 이동     | Lead Pursuit            | 적극적 접근, 조준 수렴 중  |
| $\dot{\lambda} = 0$ | LOS 고정 (충돌 경로) | <b>Collision Course</b> | WEZ 진입 직전 상태     |
| $\dot{\lambda} < 0$ | LOS가 후방 이동     | Lag Pursuit             | 오버슈트 후 또는 보수적 추적 |

**핵심 인사이트:**  $\dot{\lambda} = 0$ 은 **Collision Course** 조건이다. WEZ 진입·유지를 위해서는  $\dot{\lambda} \approx 0$ 을 목표로 heading을 조정해야 한다. 이것이 PN 유도 of 핵심이다.

대안적 계산 (적 속도 미관측 시):

$$\dot{\lambda} \approx \frac{\Delta(\text{ATA})}{\Delta t} \quad (\text{연속 2 tick의 ATA 변화율})$$

이 근사는 적 속도 관측 없이도 매 tick ata\_deg 변화량만으로 계산 가능하다.

### 10.1.3 Intercept Geometry — 각도 관계의 항등식

HCA = ATA + AA

- HCA** (Heading Crossing Angle): 두 항공기 기수 방향이 이루는 각도  $\rightarrow$  hca\_deg
- ATA** (Antenna Train Angle): 적이 나를 바라보는 각도  $\rightarrow$  ata\_deg
- AA** (Aspect Angle): 내가 적을 바라보는 각도  $\rightarrow$  aa\_deg

이 항등식으로 **세 각도 중 두 개를 알면 나머지를 역산**할 수 있다. 관측 벡터에 세 값이 모두 있으므로 cross-validation에도 사용 가능.

**Collision Course 조건** (등속 가정):

$$V_{\text{ego}} \cdot \sin(\text{ATA}) = V_{\text{enm}} \cdot \sin(\text{AA})$$

등속( ego\_vc\_kts  $\approx$  적 속도)일 때 간소화:

$$|\text{ATA}| = |\text{AA}|, \quad \text{CB} = \frac{\text{HCA}}{2}$$

10.1.4 Closure Rate 공식 — 관측 벡터에서 직접 계산

$$V_c = V_{ego} \cdot \cos(ATA) + V_{enm} \cdot \cos(180^\circ - AA)$$

이 공식으로 closure\_rate\_kts 의 물리적 의미를 분해할 수 있다.

Pursuit 유형별 Closure 효과:

| Pursuit 유형 | ATA 범위             | AA 변화 | Closure 효과        | 전술 목적             |
|------------|--------------------|-------|-------------------|-------------------|
| Lead       | ATA > 0 (기수가 적 전방) | AA 증가 | Closure 증가, 거리 감소 | 사격 솔루션 확보 (Gun)   |
| Pure       | ATA ≈ 0 (기수가 적 위치) | AA 유지 | 중간 Closure        | 레이더 락/IR 미사일 위치   |
| Lag        | ATA < 0 (기수가 적 후방) | AA 감소 | Closure 감소, 거리 증가 | 선회 공간 확보, 오버슈트 방지 |

10.1.5 선회 성능 (Turn Performance)

$$\omega = \frac{g \cdot \sqrt{n^2 - 1}}{V} \quad [\text{rad/s}], \quad R = \frac{V^2}{g \cdot \sqrt{n^2 - 1}} \quad [\text{ft}]$$

| 변수       | 의미                  | 관측 매핑                                       |
|----------|---------------------|---------------------------------------------|
| $\omega$ | 선회율 (Turn Rate)     | turn_rate_degs (°/s)                        |
| $R$      | 선회 반경 (Turn Radius) | 역산: $R = V / \omega$                        |
| $n$      | Load Factor (G 배수)  | 역산: $n = \sqrt{1 + (V \cdot \omega / g)^2}$ |
| $V$      | TAS                 | ego_vc_kts × 1.6878 (ft/s)                  |

turn\_rate\_degs 와 ego\_vc\_kts 로 현재 Load Factor를 역산할 수 있고, 이것은 기동 여유도(maneuvering margin)의 지표가 된다.

10.1.6 오버슈트 예측 — overshoot\_risk 의 수학적 기초

$$t_{\text{impact}} = \frac{\text{dist}}{V_c} \quad [\text{s}], \quad t_{\text{turn}} = \frac{ATA}{\omega_{\text{max}}} \quad [\text{s}]$$

$$t_{\text{impact}} < t_{\text{turn}} \Rightarrow \text{OVERSHOOT}$$

"적에게 도달하는 시간 < 기수를 적에게 맞추는 시간" → 기수를 돌리기 전에 적을 지나쳐버림.

175004 매치 검증 예시 (v6 vs ace, step 248):

dist = 79 ft, V\_c = 199.8 kts = 337 ft/s  
→ t\_impact = 79/337 = 0.23초

ATA = 81°, ω\_max ≈ 15°/s (corner speed)  
→ t\_turn = 81/15 = 5.4초

0.23초 << 5.4초 → 극단적 OVERSHOOT (실제 이 tick 직후 역전 발생)

10.1.7 WEZ 진입·유지 조건

$$\text{in\_WEZ} \Leftrightarrow ATA < \theta_{\text{WEZ}} \wedge \text{dist} < R_{\text{WEZ}} \wedge V_c > 0$$

현재 시스템:  $\theta_{WEZ} = 12^\circ$ ,  $R_{WEZ} = [152, 914]$  ft.

WEZ 유지를 극대화 조건:

$$\frac{d(ATA)}{dt} \approx 0 \quad \Leftrightarrow \quad \dot{\lambda} \approx 0 \quad (\text{PN으로 heading 지속 조정})$$

10.1.8 관측 벡터 → 공식 → L3 파라미터 매핑 요약

이 절의 핵심은 우리 관측 벡터  $o_t$ 의 변수만으로 모든 공식이 계산 가능하다는 것이다.

| 관측값                 | 공식에서의 역할                                      | L3 산출물             |
|---------------------|-----------------------------------------------|--------------------|
| ata_deg             | Pursuit 유형 판별, LOS rate 계산, 오버슈트 예측           | heading command    |
| aa_deg              | Intercept geometry, Collision 조건              | 기동 선택 보정           |
| distance_ft         | LOS rate 정규화, 오버슈트 판단                         | closure 전략         |
| closure_rate_kts    | PN 입력 ( $V_c$ ), WEZ 조건                       | throttle + heading |
| ego_vc_kts          | 선회 성능 계산, Collision 조건                        | speed management   |
| turn_rate_degs      | 현재 G 역산, $t_{\text{turn}}$                    | 기동 여유도             |
| hca_deg             | ATA+AA 검증, intercept triangle                 | cross-validation   |
| overshoot_risk      | $t_{\text{impact}} < t_{\text{turn}}$ 의 이진 근사 | 긴급 Lag 전환          |
| in_wez / enm_in_wez | WEZ 조건의 이진 플래그                                | 사격/회피 결정           |
| energy_diff_ft      | 기동 적극성 결정 (에너지 여유)                            | N 값 조절             |

L3 구현의 구체적 경로 (§11.14.3과 연계):

1. PN heading command:  
 $\Delta\psi = N \times (V_c / V_{\text{ego}}) \times \Delta ATA$   
→ LeadPursuit 노드의 heading 파라미터를 매 tick 조정
2. Closure 제어 (오버슈트 방지):  
 $dist < 1500ft \rightarrow N$  감소 (Lead → Lag 전환)  
 $overshoot\_risk = 1 \rightarrow N = 0$  (Pure) 또는 음수 (Lag)
3. WEZ 유지 (사격 지속):  
 $d\lambda/dt \approx 0$  유지를 목표로 N 값을 PID 방식 조절  
 $error = d\lambda/dt, \quad N(t) = N_{\text{base}} + K_p \times error + K_i \times \int error$
4. Energy-aware 조절:  
 $energy\_diff > 0$  (우세) → N 증가 허용 (적극적 Lead)  
 $energy\_diff < 0$  (열세) → N 감소 강제 (보수적 Lag)

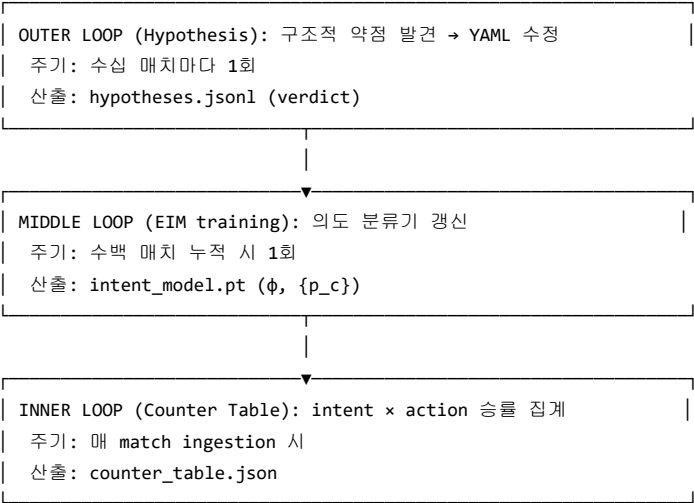
**PN 파라미터  $N$ 이 곧 L3의  $\theta$**  — 이 단일 변수가 Lead/Pure/Lag를 연속적으로 조절한다. 기존에 이산적이던 pursuit 유형 선택(L2)을 연속 파라미터 공간 (L3)으로 확장하는 열쇠다.

**적 속도 추정 문제:** 현재 관측 벡터에 적 속도( $V_{\text{enm}}$ )가 직접 포함되지 않음. §11.14.4-B의 enemy observation channel이 구현되면 정확한 LOS rate 계산 가능. 그 전까지는  $\Delta ATA / \Delta t$  근사 또는 등속 가정 ( $ego\_vc\_kts \approx enm\_vc\_kts$ )으로 대체.

# 11. 데이터 흐름 상세 — Raw 관측에서 BT 기동까지

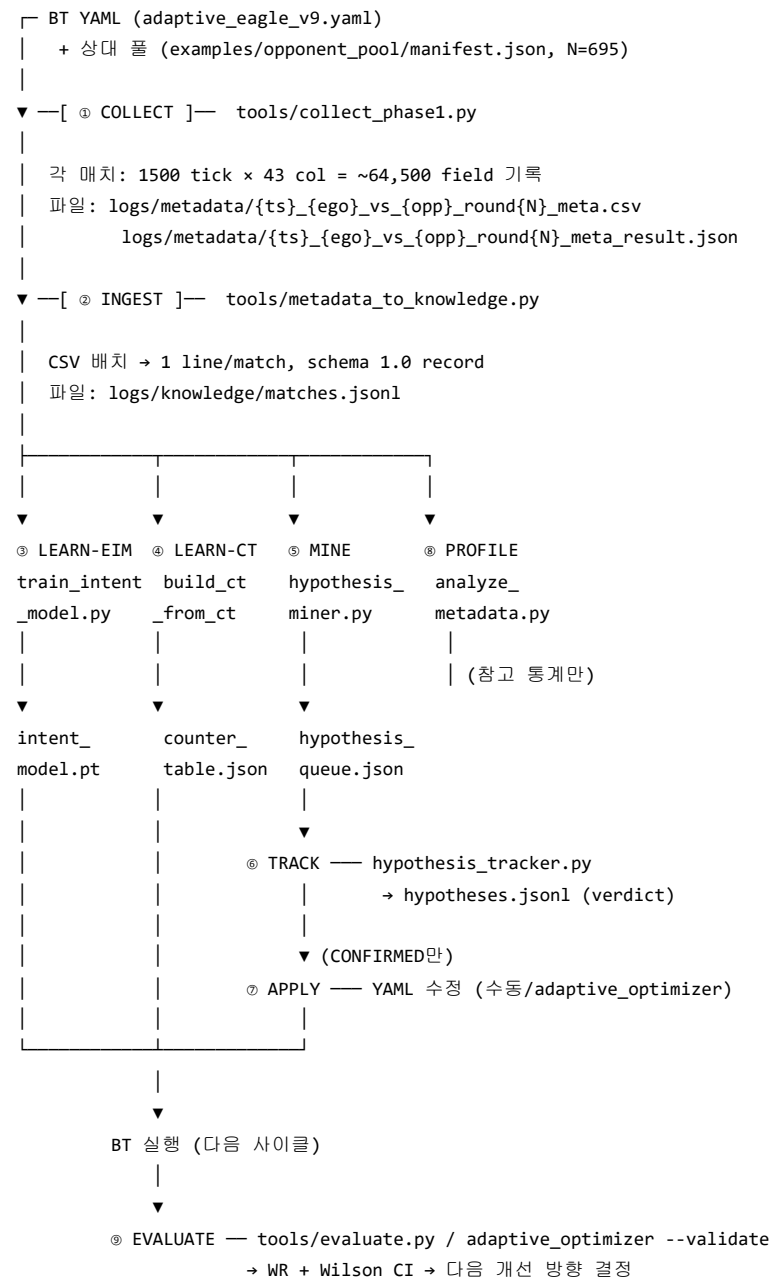
## 11.0 한 장 개요 — 3중 루프

전체 시스템은 **세 개의 중첩 루프**가 동시에 돌아간다. 모든 루프는 같은 원료(매치 CSV)를 공유하지만, 소비 방식과 산출물이 다르다.



BT는 이 세 루프의 최신 산출물을 모두 읽어서 **매 tick 즉시 의사결정**한다.

11.1 Stage 파이프라인 전체 도식



각 단계를 입력 데이터, 분석 방법, 산출물, 다음 단계 연결고리 순으로 정리한다.

11.2 Stage ① COLLECT — 매치 경험 수집

목표: 아군 BT × 상대 풀을 시뮬레이션하여 tick 단위 관측 전체를 기록.

도구: tools/collect\_phase1.py

의존: tools/metadata\_logger.py (CSV 스키마 정의)

입력

| 파라미터    | 형식        | 예시                                                |
|---------|-----------|---------------------------------------------------|
| --agent | YAML path | examples/adaptive_eagle_v9/adaptive_eagle_v9.yaml |

| 파라미터            | 형식            | 예시                                   |
|-----------------|---------------|--------------------------------------|
| --opponent-pool | manifest path | examples/opponent_pool/manifest.json |
| --rounds        | int           | 20                                   |
| --output-dir    | dir path      | logs/metadata/                       |

분석 (런타임 로깅)

시뮬레이터가 1 tick=0.2초 간격으로 진행하며, 매 tick마다 metadata\_logger.write\_row() 호출 → CSV에 43개 필드 append.

산출물

① **per-tick CSV** ( logs/metadata/{timestamp}\_{ego\_name}\_vs\_{opp\_name}\_round{N}\_meta.csv )

43 컬럼 구성:

| 그룹        | 컬럼 (개수)                                                                                                                             |
|-----------|-------------------------------------------------------------------------------------------------------------------------------------|
| 식별        | step, agent_id, tree_name, bfm_situation (4)                                                                                        |
| 기하        | distance_ft, ata_deg, aa_deg, hca_deg, relative_bearing_deg (5)                                                                     |
| 운동        | ego_altitude_ft, ego_vc_kts, specific_energy_ft, ps_fts, energy_diff_ft, closure_rate_kts, turn_rate_degs (7)                       |
| 전술<br>플래그 | in_wez, enm_in_wez, in_39_line, overshoot_risk, energy_advantage, alt_advantage, spd_advantage, tc_type, side_flag, alt_gap_ft (10) |
| 체력        | ego_health, enm_health, ego_damage_dealt, enm_damage_dealt (4)                                                                      |
| 자세        | tau_deg, roll_deg, pitch_deg (3)                                                                                                    |
| 액션        | action_alt, action_hdg, action_vel, aileron, elevator, rudder, throttle (7)                                                         |
| BT        | active_node, reward (2)                                                                                                             |
| total     | <b>43</b>                                                                                                                           |

주의: \_deg 접미어 컬럼( ata\_deg, aa\_deg, hca\_deg, relative\_bearing\_deg, tau\_deg )은 0~1 정규화 값. 도(°) 변환은 ×180. §3.1 단위 규약 참조.

② **match result JSON** ( \*\_meta\_result.json ) — 확장된 필수 필드



```

{
  "winner": "tree1|tree2|draw",
  "win_cause": "HP_ZERO|TIMEOUT|HARD_DECK|OUT_OF_BOUNDS", // 원인 명시 - 대응 전략 달라짐
  "tree1_agent": "adaptive_eagle_v9",
  "tree1_yaml_hash": "sha256:abc...", // BT 버전 고정
  "tree2_agent": "aggressive",
  "tree2_yaml_hash": "sha256:def...", // 상대 버전 고정
  "tree1_hp": 91.4,
  "tree2_hp": 0.0,
  "total_steps": 1247,
  "csv_path": "logs/metadata/..._meta.csv",
  // ▼ 재현성/root-cause 분석용 필수 필드
  "seed": 123456789, // RNG seed - 특정 매치 재현용
  "sdk_version": "0.5.5.9",
  "eim_version": "intent_model_trajectory_v3.pt",
  "ct_version": "counter_table_2026-04-20.json",
  "damage_events": [ // 원시 피격 이벤트 - 패인 분석 1순위
    {
      "tick": 172, "who_was_hit": "tree1", "damage": 4.3,
      "ata_deg": 8.6, "aa_deg": 175.7, "distance_ft": 2945,
      "closure_rate_kts": 250, "active_node_ego": "LeadPursuit",
      "active_node_enm": "SmartGunAttack"
    }
  ]
}

```

다음 단계 연결

- ② **INGEST**: CSV + JSON 쌍을 입력으로 소비
- ③ **LEARN-EIM**: CSV의 관측 28 col + active\_node 를 라벨링 원료로 사용
- ⑤ **MINE (Miner 1, 8)**: CSV에서 tick-level rigid pattern / tactical delta 추출
- ⑥ **TRACK (재현성)**: seed + \*\_yaml\_hash + \*\_version 으로 bit-exact 재실행

11.3 Stage ② INGEST — 매치 단위 요약으로 변환

목표: tick 단위 raw CSV를 매치 단위 요약 record로 압축하여 비교·통계 가능한 형태로.

도구: tools/metadata\_to\_knowledge.py

입력

| 경로                               | 형식                |
|----------------------------------|-------------------|
| logs/metadata/*_meta.csv         | 43 col per tick   |
| logs/metadata/*_meta_result.json | 7 field per match |

분석 — 매치별 집계 (v2: 데이터 기반 타당성 강화)

원칙: 모든 집계는 (a) 시계열 구조 보존, (b) 적 관점 포함, (c) 표본수 명시, (d) 주기 신호는 평균 금지.

기본 집계 (의미 있는 것만)

1. **노드 히스토그램 (ego + enemy)** — active\_node + active\_node\_enm 빈도. **상위 n개 cut 금지 — 전체 보존** (rare 노드가 중요할 수 있음).
2. **BFM 분포** — bfm\_situation 빈도 (참고용 consistency 체크, 가설 시그널로 직접 사용 금지 — coarse)
3. **불리언 tick 비율 + 표본 수** — {"wez\_pct": 30.2, "wez\_ticks": 45, "total\_ticks": 1500} 형식. 퍼센트 단독 사용 금지
4. **결과 카테고리** — hp\_diff = tree1\_hp - tree2\_hp :

```

hp_diff > +10 → WIN_DOMINANT
+2 < hp_diff ≤ +10 → WIN_MARGINAL
-2 ≤ hp_diff ≤ +2 + wez_ticks=0 → DRAW_NO_ENGAGEMENT
|hp_diff| ≤ 2 → DRAW_ENGAGED
-10 ≤ hp_diff < -2 → LOSS_MARGINAL
hp_diff < -10 → LOSS_DOMINANT

```

## 시계열 구조 보존형 집계 (신규)

5. **WEZ streak 분포** — in\_wez=1 연속 구간 길이 리스트. 평균 30% WEZ를 "45 tick 한 번" vs "3 tick씩 15번"으로 구분 가능케 함:

```

"wez_streaks": {
  "ego": [12, 3, 45, 8, 22],
  "enemy": [5, 22, 3]
}

```

6. **엔게이지먼트 phase 분절** — bfm\_situation 전이 또는 distance 교차 지점으로 phase 분리. phase별 독립 집계:

```

"engagement_phases": [
  {"start_tick":0, "end_tick":120, "phase":"MERGE",
   "ego_avg_closure":+420, "wez_ticks":0, "hp_delta_ego":0},
  {"start_tick":121, "end_tick":340, "phase":"ENGAGE",
   "ego_wez_ticks":28, "hp_delta_ego":-4.3, "ata_min":8.6}
]

```

7. **Damage event 시퀀스** — result JSON의 damage\_events 를 매치 레코드에도 복제 + 관측 컨텍스트 보강. Miner의 핵심 소스:

```

"damage_events": [
  {"tick":172, "who_was_hit":"ego", "damage":4.3,
   "ata":8.6, "dist":2945, "closure":250,
   "active_node_ego":"LeadPursuit", "active_node_enm":"SmartGunAttack",
   "phase":"MERGE", "eim_intent_at_tick":"CLOSING", "eim_conf":0.78}
]

```

## 적 관점 집계 (신규 — 이전 스키마에 0개였음)

8. **Enemy metrics** — 대칭 메트릭:

```

"enemy_metrics": {
  "enm_wez_pct", "enm_wez_ticks",
  "enm_first_wez_tick", "enm_last_wez_tick",
  "enm_damage_dealt_total",
  "enm_hp_loss_rate_per_engagement", // phase 단위
  "enm_top_nodes_full", // 전체
  "enm_bfm_pct"
}

```

## 수치 통계 — 조건부 + 안전 집계만

9. **수치 통계 (제한적)** — 주기 신호(ATA/AA)의 avg/median 금지. 안전한 것만:

- distance\_min, distance\_avg (monotone 누적 영역에서만 의미)
- closure\_max\_positive, closure\_min\_negative (극단값 기반)
- energy\_diff\_min, energy\_diff\_max
- ATA/AA는 대신 **streak** 으로: "ata<12° 연속 구간 길이 분포"

10. **Intent 조건부 메트릭** — EIM 재태깅 후 계산 (Stage ④ 참조). 최초 ingest 시엔 placeholder, EIM 모델이 있을 때 재집계:

```

"intent_conditional": {
  "CLOSING": {"n_ticks":420, "wr_during":0.75, "avg_closure":+280,
             "top_node":"SmartHighYoYo", "hp_delta":+2.1},
  "EXTENDING": {"n_ticks":180, ...},
  "ORBITING": {"n_ticks":720, ...}
}

```

## BFM 전이는 제거

이전 스키마의 BFM 전이(OBFM→DBFM)는 coarse하여 신호가 약함 → **삭제**. 대신 `node_transitions` 상위 20개로 대체:

```
"node_transitions": {  
  "SmartLeadPursuit->SmartHighYoYo": 12,  
  "SmartHighYoYo->SmartLeadPursuit": 10  
}
```

## 계산하지 말아야 할 것 (anti-patterns)

- `ata_avg` / `aa_avg` / `hca_avg` — 주기 신호의 mean은 통계적 artifact 양산
- `top_8 cutoff` — 희소 critical 노드 정보 소실
- opponent별 정규화 없는 통합 통계 — confounding 무시 (vs aggressive 편향)
- 매치 전체 구간 평균 (phase 분절 없이) — 머지/교전/이탈이 섞여 의미 상실

## 산출물

**matches.jsonl** ( `logs/knowledge/matches.jsonl` ) — 1 line = 1 match.

Schema 2.0 레코드 (v2 — 시계열/적 관점/재현성 반영):

```

{
  "schema_version": "2.0",
  "source": "metadata_csv",
  "agent": { "name", "version", "yaml_path", "yaml_hash" },
  "opponent": { "name", "yaml_path", "yaml_hash" }, // hash로 opponent 버전 고정
  "env": { "sdk_version", "eim_version", "ct_version", "seed" }, // 재현성
  "tags": { "agent_name", "agent_version", "source_tool",
            "collection_batch", "hypothesis_id", "cycle_id" },
  "outcome": {
    "winner", "win_cause": "HP_ZERO|TIMEOUT|HARD_DECK|OUT_OF_BOUNDS",
    "tree1_hp", "tree2_hp", "hp_diff", "duration_s", "n_ticks",
    "category": "WIN_DOMINANT|WIN_MARGINAL|DRAW_ENGAGED|DRAW_NO_ENGAGEMENT|LOSS_MARGINAL|LOSS_DOMINANT"
  },
  "metrics": { // 안전 집계만
    "wez_pct": 30.2, "wez_ticks": 45, "total_ticks": 1500,
    "overshoot_pct": 3.1, "overshoot_ticks": 5,
    "energy_adv_pct": 60, "alt_adv_pct": 55,
    "hp_decrease_ticks": 7,
    "distance": { "min", "avg" }, // ATA/AA avg 제거
    "closure": { "min_negative", "max_positive", "avg" },
    "energy_diff": { "min", "max" },
    "ata_wez_gate_streaks": [3, 8, 2] // ata<12 연속 구간 길이들
  },
  "wez_streaks": { "ego": [12, 3, 45, 8], "enemy": [5, 22, 3] }, // 시계열 구조
  "engagement_phases": [ // phase 분절
    { "start_tick", "end_tick", "phase": "MERGE|ENGAGE|EXTEND|REENGAGE",
      "ego_avg_closure", "wez_ticks", "hp_delta_ego", "hp_delta_enm",
      "ata_min", "dist_min" }
  ],
  "damage_events": [ // 패인 분석 1순위 소스
    { "tick", "who_was_hit", "damage", "ata", "dist", "closure",
      "active_node_ego", "active_node_enm", "phase",
      "eim_intent_at_tick", "eim_conf" }
  ],
  "node_histogram": { "ego": {...전체}, "enemy": {...전체} }, // top cut 없음
  "node_transitions": { "SmartLeadPursuit->SmartHighYoYo": 12, ... }, // 상위 20
  "bfm_pct": { "OBFM": 0.42, "DBFM": 0.18, "HABFM": 0.31, "UNKNOWN": 0.09 }, // 참고용
  "enemy_metrics": { // 적 관점 (신규)
    "enm_wez_pct", "enm_wez_ticks", "enm_first_wez_tick", "enm_last_wez_tick",
    "enm_damage_dealt_total", "enm_hp_loss_rate_per_engagement",
    "enm_top_nodes_full", "enm_bfm_pct"
  },
  "intent_conditional": { // EIM 재태깅 후 채워짐
    "CLOSING": { "n_ticks", "wr_during", "avg_closure", "top_node", "hp_delta" },
    "EXTENDING": { ... }, "ORBITING": { ... },
    "CLIMBING": { ... }, "DIVING": { ... }, "GUN_RUN": { ... }
  }
}

```

## 다음 단계 연결

- ④ **LEARN-CT**: outcome.category + top\_nodes + intent 태그로 intent×action 승률 집계
- ⑤ **MINE (Miner 2, 5)**: outcome.category 기준으로 WIN vs LOSS 분할, metrics.\* 비교 / top\_nodes 발동률 비교
- ⑧ **PROFILE**: analyze\_metadata.py 가 집계된 metrics 를 대시보드화

## 11.4 Stage ③ LEARN-EIM — 의도 분류기 학습

**목표**: "적의 관측 궤적 패턴 → 의도 클래스" 매핑 함수  $\phi + \{p_c\}$  획득.

도구: tools/train\_intent\_model.py

## 학습 데이터 스펙 — EIM이 정확히 무엇을 배우는가

입력 X: 20-tick 슬라이딩 윈도우, shape (20, 28)

28 컬럼 내역 (§3 참조):

```
continuous 14: distance_ft, ata_deg, aa_deg, hca_deg, relative_bearing_deg,
                ego_altitude_ft, ego_vc_kts, specific_energy_ft, ps_fts,
                energy_diff_ft, closure_rate_kts, turn_rate_degs, alt_gap_ft, tau_deg
binary 7:      in_wez, enm_in_wez, in_39_line, overshoot_risk,
                energy_advantage, alt_advantage, spd_advantage
BFM one-hot 7: OBFM, DBFM, HABFM, UNKNOWN, UNK_NEAR_OFF, UNK_SCISSORS, UNK_DISENGAGING
```

라벨 Y: trajectory label — 윈도우의 물리 패턴으로부터 자동 계산 (§5.3):

```
def trajectory_label(window):
    closure_mean = mean(window["closure_rate_kts"])
    dist_mean    = mean(window["distance_ft"])
    ata_mean     = mean(window["ata_deg"]) * 180
    alt_delta    = window["alt_gap_ft"][-1] - window["alt_gap_ft"][0]

    if dist_mean < 1500 and ata_mean < 20: return "GUN_RUN"
    if closure_mean > +100:                 return "CLOSING"
    if closure_mean < -100:                 return "EXTENDING"
    if alt_delta > +500:                    return "CLIMBING"
    if alt_delta < -500:                    return "DIVING"
    return "ORBITING"
```

6개 클래스 중 하나. 라벨은 BT 노드명이 아니라 관측 물리 패턴에서 나옴 → 상대의 node 구조를 몰라도 분류 가능 (§5.4).

## 모델 구조 (§5.1 재정리)

```
X: (20, 28)
↓ GRU(input=28, hidden=128, layers=2, dropout=0.1)
↓ → h_1..h_20 (각 128차원)
↓ Attention Pooling: W_e ∈ ℝ^(128,1), w_k=softmax(h_k·W_e), z = Σ w_k·h_k
↓ z (128차원)
↓ Linear(128→128) → ReLU → Dropout(0.1) → Linear(128→64)
↓ L2 normalize
↓ z_t ∈ ℝ^64, ||z_t||=1
```

## 학습 과정

1. **샘플링**: 모든 매치 CSV에서 20-tick 슬라이딩 윈도우 추출 (stride 예: 5 tick)
2. **라벨링**: 각 윈도우에 trajectory\_label() 적용
3. **훈련**:
  - Loss = ProtoNet loss (윈도우 임베딩이 자신의 프로토타입에 가깝도록, 다른 프로토타입에서 멀도록)
  - Optimizer: Adam (기본)
  - Epoch 수 / batch 등 하이퍼파라미터는 train\_intent\_model.py 참조
4. **프로토타입 생성**: 클래스 c별 학습 샘플의 평균 임베딩 → p\_c ∈ ℝ^64

## 학습 데이터 믹스 — Synthetic vs Real 비율 (2026-04-23 red team 확정)

핵심 원칙: 합성( probe\_\* , gen\_\* )과 실전 교전( aggressive , defensive , eagle\* , ...) 둘 모두 필요. 단독 사용 금지.

| 데이터 타입                           | 역할                                         | 강점              | 약점                                    |
|----------------------------------|--------------------------------------------|-----------------|---------------------------------------|
| Synthetic (probe/gen)            | 클래스 균형, minority class 확보, 초기 prototype 형성 | 순수 라벨, 균형 조정 가능 | <b>state 전이(transition) 없음</b>        |
| Real combat (표준 6 opp + agg/def) | 일반화 검증, transition 학습, fine-tuning         | 실전 분포, 복합 전략    | ORBITING trash-bin 지배, 라벨 경계 artifact |

**Synthetic 단독 금지 이유:** probe\_defensive 는 defensive 고정 → **"defensive → counter-attack" 전환** 같은 phase change를 학습하지 못함. 실전 적은 이런 전환이 빈번 → EIM 전환 순간 오분류 → 695폴 WR 하락.

**Real 단독 금지 이유:** ORBITING >60% 지배. GUN\_RUN/DIVING 같은 희소 클래스 prototype 대표성 약화. §5.5 문제 재발.

**권장 mix:**

```

훈련 세트 = 60% synthetic + 40% real
  synthetic 내부: probe_* (순수 라벨) + gen_* (semi-realistic 아키타입)
  real 내부: 표준 6 opp + aggressive/defensive (cross-version 허용)
Held-out validation = 20% of untouched real
  L3~L4 레이어 상대만 사용 — 진짜 일반화 지표

```

**평가 필수 지표** (§11.15.3 원칙 10, 11 재확인):

- **Per-class accuracy** (통합 98.8% 허위 방지)
- **Calibration** (reliability diagram —  $\text{conf} \geq 0.50$  gate가 의미 있는가)
- **Transition accuracy** — 인접 2 윈도우의 라벨이 바뀌는 구간에서 정확도 (전환 순간 오분류 측정)
- **Synthetic-trained model의 real held-out 성능** — generalization gap 측정

이 mix 원칙이 적용되지 않은 EIM 버전은 "v9의 98.8%"와 동일한 신뢰도 함정을 재생산함.

## 산출물

`intent_model.pt` — PyTorch state\_dict:

```

{
  "encoder": {GRU, Attn, Proj 가중치},
  "prototypes": {
    "CLOSING":  p_CLOSING    ∈ ℝ^64,
    "EXTENDING": p_EXTENDING ∈ ℝ^64,
    "ORBITING":  p_ORBITING   ∈ ℝ^64,
    "CLIMBING":  p_CLIMBING   ∈ ℝ^64,
    "DIVING":    p_DIVING     ∈ ℝ^64,
    "GUN_RUN":   p_GUN_RUN    ∈ ℝ^64
  },
  "metadata": {"train_date", "n_samples", "val_accuracy", ...}
}

```

## 추론 시 — 무엇을 출력하는가

입력:  $\mathbf{o}_t = [\mathbf{o}_{t-19}, \dots, \mathbf{o}_t]$  (shape 20×28)

계산:

```

z_t = φ(O_t)                ∈ ℝ^64
d_c = ‖z_t - p_c‖₂ for c ∈ 6 클래스    ∈ ℝ^6
ĉ_t = argmin_c d_c          ∈ {6 클래스}
conf_c = softmax(-d_c / τ)    ∈ [0,1]^6

```

출력:

- **intent 클래스명** (string): "CLOSING" 등
- **confidence vector** (6 원소): 각 클래스의 softmax 확률
- **distances** (6 원소): 디버깅용

## 다음 단계 연결

- ⑦ **APPLY (BT)**: 매 tick BT가 `shared_state.get_enemy_intent(ego_id)` 호출 → EIM 추론 결과 읽음
- ④ **LEARN-CT**: `matches.jsonl` 매치별로 tick 샘플에 intent 태그 → intent×action WR 집계

## 11.5 Stage ④ LEARN-CT — Counter Table 빌드

**목표**: "의도 c일 때 기동 a의 승률" 테이블 = `{CT[c]: best_a}`.

**도구**: `counter_table builder` ( `tools/metadata_to_knowledge.py` 내 함수 또는 별도 스크립트 )

### 입력

`logs/knowledge/matches.jsonl` 전체 + `intent_model.pt`

### 분석

1. **매치의 tick 샘플에 intent 태그** — 각 매치의 CSV를 다시 읽어 20-tick 윈도우 슬라이딩, EIM 추론으로 intent 부여
2. **intent 클래스 × active\_node 조합별 카운트**:

```
(intent="CLOSING", node="SmartHighYoYo") → 매치 WR 집계
(intent="CLOSING", node="LeadPursuit")   → 매치 WR 집계
...
```

3. **Wilson CI 계산** — 각 (c, a) pair의 w/D/L 에서 Wilson 95% 하한
4. **best node 선택** — `CT[c] = argmax_a WR(c, a)`, 단 최소 지원 수  $n \geq k$  조건

### 산출물

**counter\_table.json** ( `logs/knowledge/counter_table.json` ):

```
{
  "schema_version": "1.0",
  "source": "CSV ground truth (NODE_TO_INTENT mapping)",
  "n_matches": 840,
  "intent_classes": [ "GUN_ATTACK", "PURSUIT", "DEFENSIVE", "ENERGY", "NEUTRAL_CIRCLE", "NEUTRAL_SCISSORS" ],
  "entries": {
    "PURSUIT": {
      "best_node": "SmartHighYoYo",
      "wr": 1.0, "ci_lower": 0.61,
      "n": 6, "W": 6, "D": 0, "L": 0,
      "alternatives": [{"node": "SmartLowYoYo", "...": "..."}]
    },
    "CLOSING": { "...": "..."},
    "EXTENDING": { "...": "..."}
  }
}
```

(주: 현재 구현은 intent 네이밍이 문서의 6-class와 다른 node-based 6-class를 쓰는 legacy 버전이 공존함. 통일 작업 진행 중.)

## 다음 단계 연결

- ⑦ **APPLY (BT)**: TacticalLookup 노드가 `counter_table.json` 을 읽어 intent → node dispatch

11.6 Stage ⑤ MINE — 구조적 약점 탐지 (가설 생성)

목표: 현재 BT가 반복적으로 실패하는 패턴을 데이터에서 자동 추출.

도구: tools/hypothesis\_miner.py (4 miners 통합)

입력 / 출력 총괄

| Miner                   | 입력                       | 분석 단위       | 출력 패턴 수 |
|-------------------------|--------------------------|-------------|---------|
| 1 Rigid Behavior        | tick CSV                 | 20-tick 윈도우 | 5 패턴    |
| 2 Outcome Discriminator | matches.jsonl            | match       | 11 메트릭  |
| 5 Node Usage            | matches.jsonl.top_nodes  | match       | 노드별     |
| 8 Tactical Delta        | tick CSV (ego vs enm 페어) | match       | 7 피처    |

모두 hypothesis\_queue.json 에 merged.

Miner 1: Rigid Behavior (관측 불변 패턴)

20-tick 윈도우에서 같은 노드 유지 중 관측 변화 여부 검사:

| 패턴                  | 조건                                                        |
|---------------------|-----------------------------------------------------------|
| DIST_WIDENING_RIGID | distance 증가 > +500 ft + 동일 node 유지                        |
| ATA_GROWING_RIGID   | ATA 단조 증가 > +10°                                          |
| WEZ_OVERSHOOT_RISK  | 152 < dist < 1200,  ATA  < 20°, closure > 250 kts, 상태 무변화 |
| YOYO_WIDENING       | YoYo 계열 node 중 dist > +800 ft                             |
| SAME_ACTION_STREAK  | 동일 node 50 tick(=10s) 지속 + closure < 0                    |

예시 평문 해석:

- DIST\_WIDENING\_RIGID → "적이 멀어지는데도 같은 기동을 반복 → 해당 tick 근처에서 전환 조건 추가 필요"
- WEZ\_OVERSHOOT\_RISK → "사격 가능 구간에 들어갔는데 속도 감속을 안 해서 1 tick만에 빠져나감"

출력 필드:

```
{
  "candidate_id": "M1_0042",
  "source_miner": "rigid_behavior",
  "statement": "...",
  "evidence": {
    "pattern": "...",
    "match_path": "...",
    "tick": 123,
    "node": "...",
    "dist_change": 600,
    "ata_start": 10,
    "ata_end": 25,
    "closure": -50,
    "outcome": "LOSS_MARGINAL",
    "suggested_change_type": "condition",
    "priority": 0.8
  },
  "priority_score": 0.87,
  "status": "candidate"
}
```

Miner 2: Outcome Discriminator (WIN vs LOSS 메트릭 차이)

matches.jsonl.metrics 의 11개 메트릭을 WIN 매치 그룹 vs (LOSS+DRAW) 매치 그룹으로 Cohen's d 비교:

$$d = (\text{mean\_WIN} - \text{mean\_COMPARE}) / \text{pooled\_stdev}$$

임계:  $|d| \geq 0.5 \rightarrow$  가설 후보로 등록.

분석 대상 메트릭:



```
ata_avg, ata_max, aa_avg,
distance_avg, distance_min,
closure_avg, energy_diff_avg,
wez_pct, overshoot_pct,
energy_adv_pct, alt_adv_pct
```

예시 가설:

```
{
  "metric": "closure_avg",
  "win_mean": 85.3, "compare_mean": 42.1, "delta": 43.2,
  "effect_size": 0.87, "n_wins": 120, "n_compare": 45,
  "statement": "WIN 매치는 평균 closure가 43 kts 높음 (d=0.87).
               추격 속도가 승패 결정 요인일 수 있음.",
  "suggested_change_type": "threshold"
}
```

Miner 5: Node Usage Imbalance

top\_nodes 히스토그램에서 비정상 발동을 찾기.

| kind               | 조건                                              | 의미                         |
|--------------------|-------------------------------------------------|----------------------------|
| underused          | max_pct < 0.01 (모든 매치 중 최대 1% 미만)               | 아예 안 쓰이는 노드 — tree에서 제거 후보 |
| overused_in_losses | loss_pct > 0.40 AND (loss_pct - win_pct) > 0.05 | 패배 매치에서 과발동 — 조건 강화 후보     |

Miner 8: Tactical Delta

ego vs enm tick을 페어링하여 전투 지표 차이:

```
delta_turn_rate_avg = mean(ego_turn_rate - enm_turn_rate)
delta_ps_avg        = mean(ego_ps - enm_ps)
delta_cas_avg        = mean(ego_cas - enm_cas)
delta_alt_avg        = mean(ego_alt - enm_alt)
delta_energy_avg     = mean(ego_Es - enm_Es)
ego_wez_pct, enm_wez_pct (WEZ 점유율)
ego_first_wez, enm_first_wez (최초 WEZ 진입 tick)
```

임계: |Cohen's d| ≥ 0.4. priority = |d| + 0.2.

산출물

hypothesis\_queue.json — 4 miner 출력 병합.

다음 단계 연결

- ⑥ TRACK: priority\_score 내림차순으로 상위 N개 후보 선택 → 검증 라운드 실행

11.7 Stage ⑥ TRACK — 가설 검증

목표: 후보 가설의 BT 변경안을 A/B로 검증하여 유의한 개선만 채택.

도구: tools/hypothesis\_tracker.py

입력

- hypothesis\_queue.json 후보 중 선택

- baseline BT (예: v51\_rigid\_v1 )
- 변경 제안 텍스트 (예: "IsLostPursuit dist\_min\_ft=2000" )

## 분석

1. 제안된 변경 적용한 experimental BT 생성 (yaml 패치)
2. 검증 매치 실행: 기본 2 rounds × 6 opponents = 12 매치
  - opponents: eagle1, eagle2, ace, viper1, golden, alpha2
3. baseline WR과 비교:

$$\text{delta\_pp} = (\text{new\_wr} - \text{baseline\_wr}) \times 100$$

```
|delta_pp| < 5pp → INCONCLUSIVE
delta_pp ≥ 5pp → CONFIRMED
delta_pp ≤ -5pp → REFUTED
```

(주: 현재 구현은  $|\text{delta}| < 0.05$  비율 기준 — 5 percentage points)

## 산출물

**hypotheses.jsonl** — 1 line/verdict:

```
{
  "id": "H1",
  "ts": "2026-04-23T08:45:00Z",
  "statement": "IsLostPursuit의 dist 최소 조건 2000ft로 강화",
  "baseline": "v51_rigid_v1",
  "change": "IsLostPursuit dist_min_ft=2000",
  "results": [
    { "opp": "eagle1", "wins": 2, "draws": 0, "losses": 0, "hp_diff_avg": 8.3 },
    { "opp": "eagle2", "wins": 0, "draws": 1, "losses": 1, "hp_diff_avg": -3.2 }
  ],
  "totals": { "W": 8, "D": 2, "L": 2, "total": 12 },
  "wr": 0.667, "baseline_wr": 0.500,
  "delta_pp": 16.7,
  "verdict": "CONFIRMED",
  "notes": "viper1/golden OK, eagle2 fail — Pareto trade"
}
```

**situations.jsonl** (병행) — 검증 중 관측된 상황 패턴:

```
{
  "ts": "...",
  "category": "ADVANTAGE|DISADVANTAGE|STALEMATE",
  "condition": "dist > 2000 AND closure < -50",
  "freq": 34,
  "outcome_mix": { "WIN": 0.82, "DRAW": 0.12, "LOSS": 0.06 },
  "source": "hypothesis:H1"
}
```

## 다음 단계 연결

- ⑦ **APPLY**: verdict==CONFIRMED만 실제 YAML에 반영

## 11.8 Stage ⑦ APPLY — BT 구성과 실시간 활용

**목표**: 세 루프의 최신 산출물을 모두 묶어 실행 가능한 BT YAML로 통합.

입력

| 산출물                  | 출처      |
|----------------------|---------|
| intent_model.pt      | Stage ③ |
| counter_table.json   | Stage ④ |
| CONFIRMED hypotheses | Stage ⑥ |

구성 방법

1. **YAML 트리 구조 정의** — examples/adaptive\_eagle\_v9/adaptive\_eagle\_v9.yaml
  - Selector 루트 아래 우선순위 순 children:
    - a. HardDeck (순수 기하)
    - b. GunEngagement (ATA<12, dist<3000)
    - c. OffensivePursuit (기하 조건)
    - d. **TacticalLookup** ← counter\_table.json 읽음
    - e. **Counter[GUN\_RUN|CLOSING|EXTENDING|CLIMBING|DIVING|ORBITING]** ← EIM 읽음
    - f. Fallback 노드들 (기하 기반)
    - g. Default
2. **파라미터 튜닝** — CONFIRMED 가설의 변경 적용
3. **커스텀 노드 구현** — examples/adaptive\_eagle\_v9/nodes/custom\_actions.py 등

BT 실행 중 EIM/CT 활용 흐름

매 tick의 블랙보드(blackboard) 흐름:

```
[Sim Step]
└─ 관측 o_t 생성 (28개 필드)
  │
  └─ blackboard["observation"] = o_t (BT 노드들이 읽음)
  │
  └─ EIM OnlineIntentTracker 업데이트
    │
    └─ 최근 20 tick 버퍼에 o_t push
      └─ 20 tick 쌓이면 EIM 추론 → shared_state에 저장:
        shared_state["intent"][ego_id] = ("CLOSING", 0.78)

[BT Tick]
└─ 루트 Selector 내림차순 평가
  │
  └─ Condition 노드 (예: EnemyIntentIs):
    │
    │   from src.intent import shared_state
    │   intent, conf = shared_state.get_enemy_intent(ego_id)
    │   if intent == "CLOSING" and conf >= 0.50:
    │       return SUCCESS
    │
  │
  └─ TacticalLookup 노드:
    │
    │   bin_key = f"{ata}_{dist}_{closure}_{ediff}"
    │   cell = lookup[bin_key]
    │   bucket = _pick_bucket(cell, obs, closure) # intent 참고
    │   execute bucket["node"]
    │
  └─ Action 노드 (예: SmartHighYoYo):
    │
    │   obs = blackboard["observation"]
    │   alt, hdg, vel = compute_action(obs)
    │   blackboard["action"] = [alt, hdg, vel]
```

```
[Sim Step]
└─ blackboard["action"] 읽어서 기체 제어 → 다음 tick
```

산출물

examples/adaptive\_eagle\_v9/adaptive\_eagle\_v9.yaml + nodes/custom\_\*.py

다음 단계 연결

- ⑨ **EVALUATE**: 이 BT를 상대 풀에 대해 대규모 검증

11.9 Stage ⑧ PROFILE (보조) — 탐색적 분석

**목표**: 매치 데이터를 사람이 이해 가능한 지표로 정리하여 전략 결정에 사용.

**도구**: tools/analyze\_metadata.py

주요 지표

| 약어  | 의미                               |
|-----|----------------------------------|
| SAE | State-Action Entropy (노드 선택 다양성) |
| TIR | Time-In-Range (WEZ/공격 범위 체류 비율)  |
| WPP | WEZ Pass-Probability             |
| WCS | Winning Closure Signature        |
| EIP | Energy Invariant Preservation    |
| EVW | Engagement Volume per Win        |

**산출물**: 콘솔 리포트 + logs/analysis/\*.json

직접 BT를 수정하지 않지만, 다음 사이클의 **가설 방향 결정**(어느 구간/메트릭에 집중할지)에 영향.

11.10 Stage ⑨ EVALUATE — 최종 검증

**목표**: BT의 실전 성능을 통계적으로 신뢰 가능한 수준으로 측정.

**도구**:

- tools/evaluate.py (library, Wilson CI 계산)
- tools/adaptive\_optimizer.py --validate <yaml> (전체 695 풀 병렬 검증)

입력

| 항목      | 예시                                          |
|---------|---------------------------------------------|
| BT yaml | adaptive_eagle_v9.yaml                      |
| 상대 풀    | examples/opponent_pool/manifest.json (695개) |
| rounds  | 2~20                                        |

분석

- 매치 실행 (ProcessPoolExecutor 병렬)
- 상대별 W/D/L 집계
- Wilson 95% CI 계산:

CI = Wilson(W, total, z=1.96)

산출물

logs/full\_pool\_validation.json :

```
{
  "total_wr": 0.5501,
  "total_ci": [0.529, 0.571],
  "per_layer": {
    "L1": {"wr": 0.72, "ci": [0.65, 0.78], "n": 100},
    "L2": {"wr": 0.61},
    "L3": {"wr": 0.37},
    "L4": {"wr": 0.84}
  },
  "per_opponent": {
    "opp_0001": {"wr": 1.0, "W": 5, "D": 0, "L": 0}
  }
}
```

다음 단계 연결

- L3 같은 **약한 레이어** 발견 → Stage ② 데이터 확장 또는 Stage ⑤ Miner 재실행
- **전체 WR 회귀** 감지 → Stage ⑥ 직전 CONFIRMED 가설 revert
- **개별 opponent 파레토 trade-off** → 해당 opponent 집중 매치 수집 → 루프 재시작

11.11 데이터 흐름 요약 — 한 페이지 스펙

| # | 단계        | 도구                                                           | 주 입력                   | 주 산출물                                              | 핵심 필드                                                 |
|---|-----------|--------------------------------------------------------------|------------------------|----------------------------------------------------|-------------------------------------------------------|
| ① | COLLECT   | collect_phase1.py                                            | YAML pair × rounds     | *_meta.csv (43 col) + *_meta_result.json (7 field) | active_node , distance_ft , closure_rate_kts , ...    |
| ② | INGEST    | metadata_to_knowledge.py                                     | CSV+JSON               | matches.jsonl (schema 1.0)                         | outcome.category , metrics.* , top_nodes , bfm_pct    |
| ③ | LEARN-EIM | train_intent_model.py                                        | CSV (20-tick 윈도우)      | intent_model.pt                                    | encoder weights, prototypes.{CLOSING,...}             |
| ④ | LEARN-CT  | (builder)                                                    | matches.jsonl +EIM 재태깅 | counter_table.json                                 | entries.{c}.best_node , wr , ci_lower                 |
| ⑤ | MINE      | hypothesis_miner.py                                          | matches.jsonl +CSV     | hypothesis_queue.json                              | candidate_id , pattern , effect_size , priority_score |
| ⑥ | TRACK     | hypothesis_tracker.py                                        | 후보+baseline            | hypotheses.jsonl , situations.jsonl                | delta_pp , verdict , results[]                        |
| ⑦ | APPLY     | (manual+optimizer)                                           | EIM+CT+CONFIRMED       | adaptive_eagle_v9.yaml + custom nodes              | tree structure, node params                           |
| ⑧ | PROFILE   | analyze_metadata.py                                          | matches.jsonl          | analysis 리포트                                       | SAE, TIR, WPP, WCS, EIP, EVW                          |
| ⑨ | EVALUATE  | <a href="#">evaluate.py</a> / adaptive_optimizer -- validate | BT+풀                   | full_pool_validation.json                          | total_wr , per_layer , per_opponent , Wilson CI       |

11.12 "EIM은 정확히 무엇을 배우고 무엇을 출력하는가" — 한 눈 요약

학습 시

| 항목          | 구체값                                                                 |
|-------------|---------------------------------------------------------------------|
| 학습 데이터 1 샘플 | 20-tick × 28-dim tensor ( $x \in \mathbb{R}^{\{20 \times 28\}}$ )   |
| 라벨          | 6 클래스 중 하나 (trajectory rule-based)                                  |
| 라벨 출처       | trajectory_label() — 관측의 물리 통계로 자동 계산                               |
| 파라미터        | GRU(28→128, 2 layers) + Attn W_e(128→1) + Proj(128→64) + 프로토타입 6×64 |
| Loss        | ProtoNet loss (클래스 내 가깝게, 클래스 간 멀게)                                 |

추론 시

| 입력                                                     | 출력                                            |
|--------------------------------------------------------|-----------------------------------------------|
| 최근 20 tick의 관측 $o_t \in \mathbb{R}^{\{20 \times 28\}}$ | $\hat{c}_t \in \{6 \text{ 클래스}\}$ (intent 라벨) |
|                                                        | $conf_c \in [0,1]^6$ (softmax 확률)             |
|                                                        | $d_c \in \mathbb{R}^6$ (각 프로토타입과의 거리)         |

BT에서의 활용

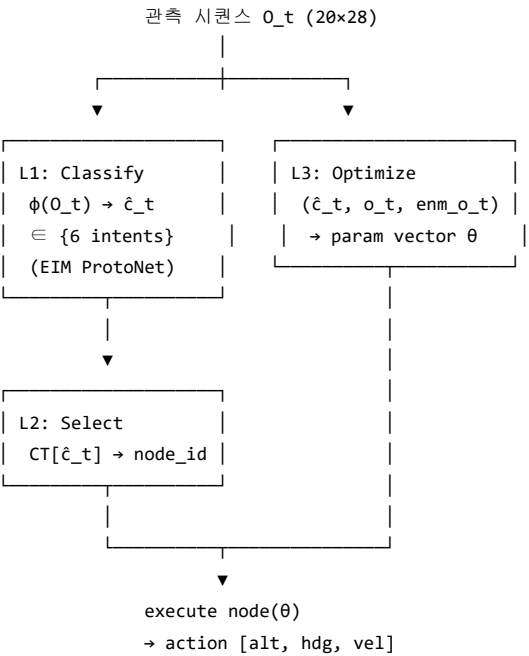
| 소비 지점                                           | 어떻게 씬                                                            |
|-------------------------------------------------|------------------------------------------------------------------|
| EnemyIntentIs(intent, min_confidence) Condition | $intent == \hat{c}_t$ AND $conf \geq min\_confidence$ 이면 SUCCESS |
| TacticalLookup Action                           | $\hat{c}_t$ 로 aggressive/defensive bucket 선택 → counter_table 참조  |

11.13 재현성 규칙 — 매번 지킬 것

- 1. 수집 배치마다 collection\_batch 태그 — matches.jsonl에 기록해 이후 추적 가능
- 2. 가설 ID 부여 — cycle\_id\_H{n} 형식. matches.jsonl의 tags.hypothesis\_id 에 기록
- 3. EIM 재학습 트리거 — 신규 matches 누적 ≥ 200 + L3 같은 약 레이어 WR < 40%
- 4. CT 재빌드 트리거 — 새 노드 추가 후 최소 10 매치/(intent, node) 쌍 확보
- 5. baseline 보존 — 현재 챔피언 BT는 삭제 금지. submissions/ 하위에 버전 디렉토리로 고정
- 6. 실험 격리 — 한 실험은 한 가지 변경만. "여러 개 동시 수정 → 단일 테스트"는 원인 분리 불가 (v8 사례: 22% 폭락)

11.14 3-Layer 의사결정 계층 — Classify / Select / Optimize

§2의 핵심 아이디어를 정식으로 확장하면 순차적 3레이어의 의사결정이다.



L1은 "무엇을 하는가?"(적 의도), L2는 "무엇으로 대응?"(BFM 선택), L3는 "어떻게?"(파라미터 튜닝).

문서의 §11.0~§11.15는 L1/L2에 집중했다. L3 없이는 adaptive BT의 세 번째 축이 빠진다.

11.14.1 L1 Classify — 이미 정의됨

| 항목     | 내용                                          |
|--------|---------------------------------------------|
| 학습 소스  | CSV 모든 매치의 20-tick 윈도우 + trajectory_label() |
| 학습 산출  | intent_model.pt (φ, {p_c})                  |
| 실시간 입력 | 최근 20 tick 자기 관측 o_t                        |
| 실시간 산출 | (ĉ_t, conf_c, d_c)                          |
| 정의 위치  | §11.4, §11.12 full spec                     |

11.14.2 L2 Select — 부분 정의됨 (확장 필요)

현재: CT[ĉ\_t] → single best node .

필요: CT[ĉ\_t, state\_bin] → node — 같은 intent라도 **state에 따라 다른 기동**. 이미 TacticalLookup 이 4D state bin (ata × dist × closure × e\_diff)을 쓰지만 문서화 부족.

보강 스키마:

```
"counter_table_v2": {
  "schema_version": "2.0",
  "bin_edges": { "ata": [0,30,60,120,180], "dist": [...], "closure": [...], "e_diff": [...] },
  "cells": {
    "CLOSING__ata0_dist1_clos2_ediff0": {
      "best_node": "SmartHighYoYo",
      "wr": 0.78, "ci_lower": 0.65, "n": 34,
      "alternatives": [...]
    },
    "CLOSING__ata1_dist1_clos2_ediff0": {
      "best_node": "SmartLeadPursuit",
      "wr": 0.82, "ci_lower": 0.70, "n": 28
    }
  }
}
```

| 학습 소스  | CSV에서 (tick state_bin, active_node, match outcome) 튜플 집계 |
|--------|----------------------------------------------------------|
| 실시간 입력 | $\hat{c}_t$ + 현재 tick의 (ata, dist, closure, e_diff)      |
| 실시간 산출 | node_id                                                  |

11.14.3 L3 Optimize — 수학적 기초 확보, 구현 대기 (2026-04-24 갱신)

정의: 선택된 기동의 파라미터  $\theta$ 를 관측 쌍  $(o_t^{\text{ego}}, o_t^{\text{enm}})$  으로부터 실시간 결정.

수학적 기초 — §10.1 BFM Guidance Law 참조: L3의 핵심 돌파구는 **Proportional Navigation(PN) 유도 법칙**이 우리 관측 벡터만으로 계산 가능하다는 발견이다(§10.1.1~§10.1.8). 이로써 L3의  $\theta$  공간이 무한→유한으로 축소된다.

PN 기반  $\theta$  파라미터화 — L3의 핵심 변수:

$$\theta_{\text{PN}} = (N, K_p, K_i)$$

| 파라미터  | 의미                  | 물리적 효과                   | 탐색 범위    |
|-------|---------------------|--------------------------|----------|
| $N$   | Navigation Constant | Lead↔Lag 연속 조절 (§10.1.1) | [0, 6]   |
| $K_p$ | LOS rate 비례 게인      | WEZ 유지 반응 속도 (§10.1.7)   | [0, 2]   |
| $K_i$ | LOS rate 적분 게인      | 정상상태 오차 보상               | [0, 0.5] |

이 3개 파라미터가 관측값의 함수로 결정되면:

$$\theta_t = f_{\text{PN}}(o_t) = \left( N(\text{dist, closure, ATA}), K_p(\text{energy\_diff}), K_i(\text{enm\_in\_wez}) \right)$$

관측 차 기반 입력:  $\delta_t = o_t^{\text{ego}} - o_t^{\text{enm}}$

- 상대 속도차, 상대 고도차, 상대 에너지차, LOS 기반 closure 등 모두  $\delta$  의 성분
- L3 의 학습 목표는  $(\delta_t, \text{node}) \rightarrow \theta \rightarrow \mathbb{E}[\Delta \text{text{hp}}_{t..t+k}] > 0$  를 최대화
- §10.1의 공식들이  $\delta \rightarrow \theta$  매핑의 **basis function 역할**

⚠️ **중요 — prescriptive signature 금지:**

- "tail chase가 정답이다", "AA가 감소해야 한다" 같은 **a priori 성공 패턴 지정 금지**
- §10.1의 공식은 **탐색 공간 설계 도구**이지 최적해가 아님 (§10.1 원칙 box 참조)
- 어떤  $\delta$  영역에서 어떤  $\theta$ 가 이기는지는 **Miner/Tracker가 데이터에서 발견할 사항**
- 단일 WIN match 관찰 → 일반 법칙 격상 = 단편 수정 anti-pattern (feedback\_no\_piecemeal\_fixes 위반)
- Reference benchmark(예: v6 vs ace의 tail-chase 매치)는 **시각적 예시**로만 활용, 규범으로 격상 금지

Miner 9 "Param Efficacy" 가설 후보 (§10.1 공식 기반으로 구체화):



| 가설                                        | §10.1 근거                                                  | 탐색 변수                       |
|-------------------------------------------|-----------------------------------------------------------|-----------------------------|
| SmartHighYoYo: 고 closure 시 pitch-up 지연 단축 | §10.1.6 오버슈트 예측 ( $t_{\text{impact}} < t_{\text{turn}}$ ) | $N$ 값 vs closure_rate 구간    |
| SmartLeadPursuit: 근거리에서 N 감소 → Lag 전환     | §10.1.1 PN ( $N \downarrow \rightarrow \text{Lag}$ )      | $N$ vs distance_ft 구간       |
| SmartBreakTurn: 에너지 열세 시 max-G 즉발         | §10.1.5 선회 성능 ( $\omega$ 극대화)                             | Break timing vs energy_diff |
| WEZ 유지: $\dot{\lambda} \approx 0$ PID 제어  | §10.1.2 LOS rate + §10.1.7 WEZ 조건                         | $K_p, K_i$ vs WEZ 체류 시간     |

**학습 소스 — 현재 데이터에서 뽑을 수 있는 것:**

```

(state_t, node,  $\theta_t$ , state_{t+k},  $\Delta hp_{\{t,t+k\}}$ )
- state_t:      28 obs + 28 enemy obs (대칭)
- node:         active_node_t
-  $\theta_t$ :       action_alt/hdg/vel (현재는 3차원 명령. 노드 내부 gain 로그 없음)
-  $\Delta hp$ :      t..t+k 구간 아군/적 HP 변화
+ NEW: §10.1 파생값 (매 tick 계산 가능):
-  $\lambda_{\text{dot}}$ :   $\Delta \text{ATA} / \Delta t$  (LOS rate 근사, §10.1.2)
- t_impact:     dist /  $V_c$  (§10.1.6)
- t_turn:      ATA /  $\omega$  (§10.1.6)
- n_load:       $\sqrt{1 + (V \cdot \omega / g)^2}$  (§10.1.5)

```

**실시간 입력:** node\_id + o\_t + enm\_o\_t + §10.1 파생값  
**실시간 산출:**  $\theta = f(\text{node\_id}, o_t, \text{enm\_o\_t})$  (노드 내부 계산, PN 기반)

### 11.14.4 L3 구현을 위한 데이터 파이프라인 요구사항

**A. 노드 내부 파라미터 로깅 —** 현재 CSV는 최종 명령(alt/hdg/vel)만. 내부 gain/threshold 추가:

```

new CSV columns (optional, per active node):
node_param_0..node_param_N (TUNABLE_PARAMS 런타임 값)

```

**B. Adversarial observation channel —** BT 노드에서 enm\_o\_t 를 읽을 수 있도록 blackboard 확장:

```

# src/intent/shared_state.py 확장
shared_state["enemy_observation"][ego_id] = enm_obs # per tick

```

현재는 자기 obs만 blackboard에, 적 obs는 간접(aa\_deg 등)으로만 접근.

**C. State-conditional param 학습 — Miner 9 "Param Efficacy":**

```

입력: CSV (state_bin, node, param_vector,  $\Delta hp_{\text{after\_k\_ticks}}$ )
분석: 같은 (state_bin, node) 내 param vector별 conditional WR
출력: "state=X + node=Y일 때  $\theta=[1.3,0.5]$  WR 78%,  $\theta=[0.9,0.5]$  WR 42%"

```

**D. Param lookup table —** L2 CT와 병렬로:

```

"param_table": {
  "SmartHighYoYo__closing_high_closure": {
    "params": {"pitch_gain": 1.3, "roll_delay": 0.2},
    "wr": 0.78, "n": 50
  }
}

```

**E. BFM 불변 규칙의 데이터 검증 —** 현재 SmartLeadPursuit의 5법칙(custom\_actions.py)이 rule-of-thumb. 각 threshold가 데이터 검증 통과했는지 기록:

- dist\_widen\_thresh\_ft=30 → 어떤 match에서 최적이었는가? 가설 기록 없음 → 임의값 의심.

11.14.5 Opponent-specific adaptation

매 매치 첫 N tick (예: N=100)을 **opponent profiling phase**로:

```
Tick 0..100      : NEUTRAL probe mode
                  → 20-tick 윈도우 EIM 여러 번 추론 → 적의 intent 분포
                  → bfm_profile = {CLOSING:0.6, ORBITING:0.3, EXTENDING:0.1}

Tick 100+        : 이 profile에 맞는 variant 로드
                  → CT_variant[bfm_profile]
                  → param_table_variant[bfm_profile]
```

구현 요구:

- Stage ⑦ APPLY에 opponent profiler 노드 추가
- CT를 opponent-archetype별 분할 (L1~L6 풀별)
- Stage ④ 학습이 archetype별 집계 지원

11.14.6 In-match regret signal (실시간 자가 교정)

BT가 stateless tick tree이지만, shared\_state 에 **running regret**을 유지하면 상태 도입 가능:

```
shared_state["regret"][ego_id] = {
    "recent_hp_loss_rate": ...,      # 최근 100 tick HP 손실률
    "wez_miss_streak":    ...,      # WEZ 진입 없이 지난 tick 수
    "intent_prediction_churn": ...,  # EIM intent 변화 빈도
}
```

BT에 새 Condition IsLosingEngagement(threshold) 추가 → 참이면 파괴적 fallback(ExtensionBreak 등) 강제 발동.

학습 소스: 매치 결과와 regret 시계열의 상관 — **Miner 10 "Regret→Outcome"**.

11.14.7 Reward 정의 명문화 (1차 추출, 2026-04-23)

출처: src/simulation/envs/JSBSim/configs/1v1/NoWeapon/bt\_vs\_bt.yaml + reward\_functions/\_\_init\_\_.py .

구현체는 .cp314-win\_amd64.pyd 바이너리라 내부 공식 직접 열람 불가 — **구성 기반 reconstruction**.

**Reward = Σ (각 컴포넌트)**, 매 agent\_interaction\_step(= 12 sim\_step = 0.2초)마다 계산:

| 컴포넌트               | 스케일      | 활성 조건              | 물리 의미                                                                                                      |
|--------------------|----------|--------------------|------------------------------------------------------------------------------------------------------------|
| PostureReward      | 100.0    | always             | WEZ 중심 지향성 + 거리. orientation=v2, range=v3, target_dist=0.5 (500m=1640ft, WEZ 중심). potential=true (shaping) |
| AltitudeReward     | (Kv=0.5) | always             | safe_alt=1500m / danger_alt=500m (Hard Deck 근처). 저고도 진입 시 패널티                                              |
| EventDrivenReward  | 1.0      | 이벤트 시              | kill/hit/crash 등 이산 이벤트. potential=true                                                                    |
| HeadingReward      | —        | 미사용 (missile-only) | 1v1/NoWeapon/bt_vs_bt에선 import만 됨                                                                          |
| ShootPenaltyReward | —        | 미사용                | missile 시나리오용                                                                                              |

**도미넌트 항은 PostureReward (scale 100)**. 즉 reward ≈ 100 × (WEZ 내 지향 품질) + (고도 보정) + (이벤트 보너스/페널티).

potential=true 는 potential-based shaping (Ng et al. 1999)로, 최적 정책 변경 없이 수렴만 가속 — aggregate 해석 시 shaping term이 평균 0에 수렴하는 성질 이용 가능.

현 시점 집계 시 주의:

- CSV의 reward 컬럼은 이 합산값. 단독 "좋고 나쁨" 판단은 스케일 차이로 왜곡됨 → 항상 sign + magnitude 같이 본다.
- Miner 2가 energy\_diff\_avg 를 WIN/LOSS 간 비교할 때, 이 reward와 독립된 지표로 쓰는 것이 더 안전 (reward는 이미 posture에 의해 결정됨).

미확인 항목 (후속 확인 필요):

- 각 컴포넌트의 **정확한 수식**은 .pyd 내부 — 디스어셈블 없이는 확정 불가. 공식 RL 논문/SDK 변경 로그 참고 필요.
- `potential=true` 일 때 첫/마지막 tick 처리 방식
- `range_version=v3` 의 거리 감쇠 함수 (지수? 가우시안?)

**L3 optimization의 objective:** 위 reward를 직접 사용하기보단, **매치 단위 aggregate metric** 사용이 안전:

- HP 보존율 ( `tree1_hp / 100` )
- 승률 (Wilson 하한)
- WEZ 점유 시간 ( `wez_pct` )
- 매치 길이 대비 HP 교환비

reward 컬럼의 per-tick 신호는 high-frequency noise가 많아 L3의  $\theta$  선택 시 직접 비교 부적합. aggregate 쪽이 통계적 안정성 높음.

### 11.14.8 3-Layer 구현 체크리스트 — 데이터 gap 제로 보증

- [ ] Reward 정의 문서에 명시됨 (11.14.7)
- [ ] CSV에 노드 내부 param 컬럼 추가됨 (11.14.4-A)
- [ ] shared\_state에 enemy\_observation 노출됨 (11.14.4-B)
- [ ] CT가 state-bin 단위로 확장됨 (11.14.2)
- [ ] Miner 9 "Param Efficacy" 구현됨 (11.14.4-C)
- [ ] param\_table 스키마 정의됨 (11.14.4-D)
- [ ] BFM 불변 법칙 threshold가 데이터 검증 기록 보유 (11.14.4-E)
- [ ] Opponent profiler phase BT에 추가됨 (11.14.5)
- [ ] shared\_state.regret + IsLosingEngagement Condition (11.14.6)

하나라도 미완이면 "3-layer adaptive BT"라고 부르지 말 것 — 현재는 **1.5-layer (L1 + 부분 L2)**.

## 11.15 데이터 설계 원칙 — 논리 gap 제거 (횡단 concern)

개별 Stage 스펙과 무관하게 **전체 파이프라인에서 반드시 지켜야 할 정합성 규칙**. 한 개라도 위반되면 그 아래 모든 분석 결과의 타당성이 흔들린다.

### 11.15.1 통계 유효성

1. **주기 신호 평균 금지** — ATA/AA/HCA/relative\_bearing은 0~360° 주기 신호. `mean(ATA)` 는 "항상 90°"와 "0°↔180° 진동"을 같은 값으로 표시. 대신 **streak 분포, 특정 역치 below 비율, circular statistics** 사용.
2. **Cohen's d는 per-opponent stratification 후** — WIN 매치의 closure가 높은 것이 "closure가 이긴다"는 증거가 되려면 **같은 상대**에서 비교해야 함 (simpson's paradox). 통합 풀 비교는 confounding 1순위.

```
nope: d = (mean_WIN_all_opp - mean_LOSS_all_opp) /  $\sigma$ 
ok:   per opponent로 d 계산 → 상대별 d 분포 검토
```

3. **Multiple testing correction** — Miner 2가 11 메트릭 × 다수 비교 →  $\alpha=0.05$  naïve 임계는 FDR 폭주. 최소 Benjamini-Hochberg 적용 또는 `p_adj < 0.05` 기록. 현재 `|d| ≥ 0.5` 임계는 effect size일 뿐 significance가 아님.
4. **표본수 항상 동반** — 모든 비율/평균/분포에 `n` 또는 `n_ticks` 동반 필수. `wez_pct=30%` 단독은 정보량 0 (1500틱 중 30% vs 50틱 중 30% 완전 다름).
5. **Wilson CI 하한으로 정렬** — "WR 100% (N=2)" > "WR 80% (N=100)" 순으로 정렬되면 완전히 잘못. CT 빌드와 `best_node` 선택은 Wilson 하한 기준.

### 11.15.2 재현성 & 버전 고정

6. **매치 단위 fingerprint** — result JSON의 `seed`, `tree1/2_yaml_hash`, `sdk_version`, `eim_version`, `ct_version` 이 모두 채워져야 재현 가능. 하나라도 빠지면 해당 매치는 debugging 용도로만 사용.
7. **opponent YAML 해시 = 버전 ID** — 풀의 상대들도 유지보수 중 변경됨. 해시로 고정 안 하면 "같은 aggressive"가 실제 다른 BT일 수 있음 → 시계열 통계 오염.
8. **baseline snapshot 영구 보존** — 현재 챔피언은 `submissions/{name}/{yaml_hash}/` 구조로 고정. 덮어쓰기 금지. 회귀 시 비교 기준.

### 11.15.3 Label/모델 품질 (EIM 직결)

9. **Trajectory label 검증 루프** — §5.3 `trajectory_label()` 은 **휴리스틱**. EIM이 이걸 ground truth로 학습하면 "휴리스틱의 한계"가 모델 한계가 됨. 주기적 검증:
- Hold-out 세트에서 EIM 예측과 rule-based 라벨 일치율 측정
  - 불일치 케이스 수동 검토 (복합 패턴, 경계 구간)
  - §5.5 ORBITING trash-bin 같은 구조적 한계는 별도 클래스로 분리 검토
10. **클래스 불균형 보고** — ORBITING은 "나머지" 조건이라 지배적 가능성 높음. 학습 시 다음을 명시 보고:
- 각 클래스 샘플 수
  - Class weight 또는 oversampling 적용 여부
  - Validation 시 **per-class accuracy** (통합 accuracy 98.8%가 minor class에서 30%일 수도)
11. **EIM confidence calibration** —  $\text{conf} \geq 0.50$  gate를 쓸 거면, confidence가 실제 correctness와 정합하는지 확인 (reliability diagram). calibration 안 되어 있으면  $\text{conf}=0.6$  이 실제로 60%와 다른 정확도.

### 11.15.4 Causality vs Correlation

12. **Miner 가설 = 관찰된 상관, 인과 아님** — "WIN 매치는 closure가 높더라" ≠ "closure를 높이면 WIN". 가설은 **반드시 ⑥ TRACK의 A/B로만 검증**. 관찰 단계에서 "causes"/"때문에" 언어 금지.
13. **Paired comparison 우선** — Counter Table의 "(intent=X, node=A) vs (intent=X, node=B)" 비교는 같은 intent cell 내 쌍으로만. intent cross-over 비교는 confounding.
14. **시계열 인과 분리** — "WIN 매치에 SmartHighYoYo 많음" vs "SmartHighYoYo 때문에 WIN" — 후자 주장하려면 SARS 인과 구조 필요: (state\_t, action, state\_{t+k}, reward) . 현재 데이터 구조는 지원함 (CSV의 tick 순서) but 분석 스크립트가 활용해야 함.

### 11.15.5 선택 편향 방지

15. **DRAW\_NO\_ENGAGEMENT** 는 "노이즈 아닌 1차 신호" — 이 매치들을 분석에서 제거하거나 LOSS로 묶는 것은 **금지**. 교전 실패 자체가 가장 풍부한 가설 원천이다.

**NO\_ENGAGEMENT가 드러내는 것:**

- **BT chase 조건 결함**: 우리 BT가 defensive 상대를 못 따라잡음 → OffensivePursuit 조건 강화 후보
- **상대 전술 분류**: 교전 회피 자체가 상대의 전략 (passive draw 유도)
- **EIM 실패 패턴**: WEZ 미진입 → intent 대부분 ORBITING trash-bin으로 몰림
- **초기 조건 편향**: 특정 방향/고도 조합에서 NO\_ENGAGEMENT 급증 가능
- **가장 actionable 가설**: "vs X 60% NO\_ENGAGEMENT → chase trigger 강화" 같은 직접 가설

**올바른 취급:**

- Miner 결과에 NO\_ENGAGEMENT 전용 섹션 분리 (WIN/LOSS 비교와 병행)
- Dedupe 시 `occurrence_count` 필드 보존 — "vs X에서 15/20 NO\_ENGAGEMENT" 같은 빈도 분석 가능
- 각 상대 × BT 버전 조합별 NO\_ENGAGEMENT 비율 tracking → 개선 우선순위
- EIM 학습 시 NO\_ENGAGEMENT 샘플의 label 재검토 (대부분 "ORBITING"으로 라벨되지만 실제로는 phase 미진입)

**Miner 2 처리 방침**: WIN vs (LOSS + ENGAGED\_DRAW)로 비교. NO\_ENGAGEMENT는 **독립 카테고리**로 별도 Miner 2.5 실행.

16. **Hypothesis 검증 상대 풀 고정** — TRACK 시 `eagle1..alpha2` 6명만 쓰면 이 풀에 과적합한 개선만 CONFIRMED → 695풀에서 회귀. 최소한 "6명 검증 + L3 subset 5명 확인" 병행.
17. **Reward 정의 명문화** — CSV `reward` 컬럼의 정의가 문서 어디에도 없음. 집계하기 전 `src/match/result.pyd` 또는 `simulator wrapper`에서 정의 추출해서 이 섹션에 명시 필요. 정의 없는 수치는 집계 금지.

### 11.15.6 요약 — 데이터 결함 검출 체크리스트

새 매치 배치가 들어올 때 자동 수행:

- [ ] `result` JSON 모든 매치에 `seed + yaml_hash + version` 있음
- [ ] `damage_events` 비어있지 않음 (교전 매치의 경우)
- [ ] 각 매치의 `n_ticks` ≥ 최소 엔게이지먼트 임계 (예: 200)
- [ ] `outcome.category` 분포가 극단 쏠림 없음 (WIN만 100%면 상대가 너무 약함 시그널)
- [ ] `active_node` 중 NULL 또는 UNKNOWN 비율 < 5%
- [ ] EIM intent 분포 - ORBITING만 80%+면 trash-bin 문제 의심
- [ ] per-opponent n 균등 (한 상대 과대표집 금지)

하나라도 실패 시 해당 배치는 신뢰할 수 없음 → 재수집 또는 격리.

11.16 현재 상태와 다음 할 일 (template)

매 사이클 시작 시 다음을 작성:

```
## Cycle N - {날짜}

### Baseline
- BT version: _____
- 695 풀 WR: _____ (Wilson CI _____)
- L3 WR: _____ (악점 레이어)

### 이번 사이클 목표
- 메인 가설: _____
- 측정 지표: _____
- Go/No-go 기준: _____

### 실행 계획
1. [ ] ① COLLECT - rounds _____ x opponents _____
2. [ ] ② INGEST - matches.jsonl append
3. [ ] ③ MINE - 상위 priority 가설 _____개 선택
4. [ ] ④ TRACK - {hypothesis_id} 검증 12 매치
5. [ ] ⑤ APPLY (if CONFIRMED) - YAML 수정
6. [ ] ⑥ EVALUATE - 695 풀 재검증

### 결과 / 학습
- ...
```

12. 레거시 데이터 처리 전략 — "먼저 모은 것부터 쓴다"

12.1 현재 데이터 자산 인벤토리 (2026-04-23)

| 자원                                  | 개수    | 처리 상태          |
|-------------------------------------|-------|----------------|
| Raw *_meta.csv (tick-level)         | 2,400 | 대부분 미처리        |
| Raw *_meta_result.json              | 2,399 | 대부분 미처리        |
| 매치 배치 디렉토리                          | 28    | v5~v9 다양한 버전   |
| matches.jsonl (ingested)            | 126   | CSV 대비 5.25%   |
| hypotheses.jsonl                    | 12    | A/B verdict 포함 |
| situations.jsonl                    | 5     | 상황 패턴          |
| counter_table.json + v3/v4 proposal | 4 버전  | 통합·검증 미완       |
| tactical_patterns_v1~v3.json        | 3     | 통합 미완          |
| coverage_gaps.json                  | 1     | 미활용            |

결론: 신규 수집하기 전에 기존 2,400 CSV의 95%가 knowledge base에 들어가지 않음. 먼저 이것을 정리해야 한다.

12.2 왜 "신규 수집보다 legacy 처리 우선"이 맞는가

- 표본 수 확보 비용: 매치 1건 ≈ 5초. 2,400건 신규 수집 ≈ 3시간 compute. 하지만 ingest는 수분.
- 버전 다양성: 레거시는 v5~v9까지 다양한 BT가 섞임 → BT 변천의 비교군 이미 보유. 신규는 단일 BT 데이터만 쌓음.
- 상대 다양성: 배치별로 다른 상대 풀 포함 → 695 서브셋 자연스럽게 샘플링됨.

- **Sunk cost 회복:** 이미 compute 비용 지불했으므로, 안 쓰면 100% 낭비.

## 12.3 레거시 데이터의 제약 (red team)

- \*\_meta\_result.json 이 현행(v2) 스키마 — **seed, yaml\_hash, damage\_events, win\_cause** 누락. 이는 **이후 수집부터** 적용 가능, 과거 데이터엔 소급 불가.
- 과거 BT에서 쓴 **active\_node** 이름과 현행 노드명이 다를 수 있음 → 매핑 테이블 필요.
- BT YAML 버전이 디렉토리 이름으로만 기록 → 실제 YAML 복원 불가능한 배치 있음.
- 과거 matches.jsonl.bak\_schema0 존재 → schema 0→1 마이그레이션 흔적. **schema 1→2 마이그레이션도 필요** (§11.3).

이 제약을 무시하면 안 되지만, "**재현 불가 과거**"와 "**분석 가능한 통계**"는 다르다. Intent × action WR 집계, BFM 전이, 일반 패턴 분석은 여전히 가능.

## 12.4 Legacy-first 처리 순서 (v2 — 2026-04-23 사용자 피드백 반영)

**핵심 원칙:** 학습(EIM/CT 재학습) **전에** 데이터 기반 분석이 선행. 분석은 **상대 하나씩 순차** — 절대 all-at-once miner로 전체 데이터 싹 쓸지 말 것. Coverage gate 통과 시에만 학습 단계 진행.

### Phase A — Bulk ingest (완료 2026-04-23)

1. tools/metadata\_to\_knowledge.py 를 logs/matches/\*\*/ + logs/metadata/ 에 batch 실행 ✓
2. **결과:** matches.jsonl 126 → 1,271 (10배). schema 0 baseline + schema 1 추가분 혼합 상태.

### Phase B — Knowledge base 1차 정리 (완료 2026-04-23)

3. Dedupe v2: (agent, opp, hp1, hp2, n\_ticks) signature. occurrence\_count + origin\_batches 필드 보존.
4. **결과:** 1,271 → 538 unique. 최고 중복 117x = (v9 vs aggressive NO\_ENGAGEMENT). 이 자체가 가설 소스.
5. 출력: matches\_dedupe\_v2.jsonl
6. 남은 작업: schema 1→2 마이그레이션(enemy\_metrics, wez\_streaks, engagement\_phases, damage\_events 재계산) + counter\_table\*.json 정리.

### Phase C — Per-Opponent Analysis (OPA) — 상대 하나씩 순차, OPPONENT-CENTRIC

7. **Opponent-centric framing** (2026-04-23 red team 교정):

- opp=X 매치는 **우리 BT 버전과 무관하게 모두 사용**. v6h5c vs X, v9 vs X, gen\_\* vs X 전부 같은 상대 데이터로 집계
- 이유: EIM은 "적의 관측 궤적 패턴"을 분류. 우리 BT 변경과 무관하게 **적의 관측은 동일**
- 이 프레이밍이 cross-version A/B 비교도 자연스럽게 제공 (v6h5c vs v9 on 같은 opp)

8. **분석 순서** (2026-04-23 red team: synthetic vs real 역할 분담):

#### Stage C1 — Synthetic 상대 먼저 (클래스 baseline 수립)

- probe\_wez, probe\_defensive, probe\_energy, probe\_obfm, probe\_habfm (각 21 raw) → WEZ/에너지/공격/정면 각각의 **순수 라벨 baseline**
- gen\_rush, gen\_evader, gen\_gunfighter, gen\_breaker, gen\_energy, gen\_midrange, gen\_allbranch (각 18 raw) → 아키타입별 거동
- 기대 산출: "CLOSING이 나왔을 때 정확히 어떤 관측 특성인가"의 canonical 정의
- Miner 1 (Rigid Behavior) 여기서 가장 clean하게 작동 (노이즈 최소화)

#### Stage C2 — Real 교전 상대 (일반화 + transition 분석)

- aggressive (394 raw, 68 unique) — 데이터 최다, transition 풍부
- defensive (374 raw, 84 unique)
- 표준 6 opp: eagle1, ace, viper1, golden (각 50), eagle2, alpha2 (각 21)
- 기대 산출: "실전에서 intent 전환이 언제 일어나나", "NO\_ENGAGEMENT 원인"
- Miner 8 (Tactical Delta) + NO\_ENGAGEMENT 전용 분석

#### Stage C3 — Cross comparison

- Stage C1의 순수 클래스 baseline vs Stage C2의 실전 분포 비교
- **라벨 drift 감지:** synthetic에서 CLOSING 라벨 받은 윈도우 특성 vs real에서 CLOSING 윈도우 특성
- 차이 크면 → Phase E EIM 재학습 시 이 구간을 class-weighted 강화

9. 각 상대별로:

- Miner 1/2/5/8 실행 **JUST ON THIS OPPONENT's subset, across ALL agent versions**
- NO\_ENGAGEMENT 전용 분석 섹션 (§11.15.5 원칙 15)
- **Cross-version diff:** 같은 상대에 대해 v6h5c vs v9 어떻게 달라졌나?
- 인간 review: 어떤 패턴? 어떤 gap?
- 가설 후보 → hypothesis\_queue.json에 opponent 태그로 추가
- 다음 상대로 이동 전 **findings 문서화 필수** ( logs/analysis/opponent\_{name}.md )

10. 매 상대 완료 후 coverage 맵 갱신 (아래 Phase D 참조)

Phase D — Coverage 분석 (gate)

10. State bin 커버리지 map: (intent\_class × state\_bin) 조합별 n 집계

- 임계: 각 셀  $n \geq 10$ 이어야 CT<sup>2</sup> 빌드 신뢰 가능

- gap 셀 목록 → Phase F 수집 타겟

11. Opponent 커버리지 map: L1~L6 layer별 표본 수

- v6h2의 L3 최악(36.9%) 재확인 — L3 상대 데이터 충분한가?

12. EIM 입력 커버리지: trajectory\_label 클래스 분포

- ORBITING 비율 체크. >60%면 class imbalance 이슈

13. Coverage pass 기준:

- 핵심 intent (CLOSING, EXTENDING, GUN\_RUN, DIVING) 각 클래스당  $n \geq 200$

- L3 layer 상대 최소 5개 × 각  $n \geq 20$

- NO\_ENGAGEMENT 비율 pair별 프로파일 작성됨

14. Coverage pass 시 → Phase E, 미달 시 → Phase F

Phase E — EIM 재학습 (coverage gate 통과 시)

15. 학습 데이터 준비: matches\_dedupe\_v2에서 CSV 재로드 → 20-tick 윈도우

16. Class-weighted loss 또는 oversampling (Phase D에서 ORBITING 편향 확인 시)

17. 학습 + held-out 검증:

- **per-class accuracy** 리포트 (통합 98.8% 대신 6개 클래스 각각)

- Reliability diagram (calibration 검증)

- Label audit — rule-based label과 EIM 예측 불일치 케이스 수동 검토

18. 기존 모델 대비 per-class 개선 없으면 revert

Phase F — 신규 상대 one-at-a-time 확장 (coverage gap 해소용)

19. Phase D의 gap map에서 가장 부족한 L 레이어의 상대 1명 선택

20. 충분한 round로 수집 (예: 20R × 2-3 initial condition)

21. Phase A→B→C→D 다시 실행 (해당 상대만)

22. **Gate 체크**: vs 해당 상대 승률 변화 + 695폴 회귀 없음 확인

- 회귀 있으면 해당 상대 데이터만 추가, BT 변경 금지

- 회귀 없으면 BT 개선도 동시 적용 가능

23. 다음 상대로 이동

12.4.1 "상대 하나씩" 이 맞는 이유 (red team 검증)

| 관점             | 상대 one-at-a-time             | 전체 bulk            |
|----------------|------------------------------|--------------------|
| 통계적 신호         | 같은 상대 50+ 매치 → 분산 낮음, 가설 구체적 | 평균 2 매치/상대 → 신호 약함 |
| Diagnosability | "vs X 실패 원인 = Y" 도출 가능       | "뭐가 이기고 지는지" 수준    |
| Overfitting    | 상대별 gate로 제어 가능              | 전체 과적합 감지 어려움      |
| v8 22% 폭락 재발   | per-opp gate로 차단             | bulk 수정에 여전히 취약    |
| 인력 소모          | 더 많음 (per-opp 리뷰)            | 적음                 |
| 최종 WR 견고성      | 높음                           | 낮음                 |

**결론**: 인력 대신 견고성 선택. v8 사례(한 번에 수정해서 폭락)의 재발 방지.

12.5 Legacy 처리 후 기대 효과

| 지표                              | 현재  | Phase C 완료 후 (추정)       |
|---------------------------------|-----|-------------------------|
| matches.jsonl 행수                | 126 | <b>2,400</b> (19배)      |
| 신뢰 가능한 CT cells ( $n \geq 10$ ) | 수십  | 수백~천 (state-bin 분할 후에도) |

| 지표              | 현재       | Phase C 완료 후 (추정) |
|-----------------|----------|-------------------|
| 가설 풀            | 12       | 100+ 후보           |
| per-opponent 통계 | 편향된 6~7개 | 수십~수백 opp         |
| 695폴 이해도        | 표면       | 레이어별 약점 구체화       |

이 규모의 데이터에서 **Miner 2/5/8이 즉시 실질적 가설을 쏟아낼 가능성이 크다**. 신규 수집은 그 가설들의 검증 단계(Stage ⑥ TRACK)에서 필요.

## 12.6 Phase A 스크립트 초안 (참고)

```
# bulk ingest - dry run 먼저
python tools/metadata_to_knowledge.py \
  --input-dir "logs/matches/**/" \
  --output logs/knowledge/matches_v2.jsonl \
  --schema 2.0 \
  --batch-tag legacy_bulk_2026_04 \
  --dry-run

# 실제 실행
python tools/metadata_to_knowledge.py \
  --input-dir "logs/matches/**/" \
  --output logs/knowledge/matches_v2.jsonl \
  --schema 2.0 \
  --batch-tag legacy_bulk_2026_04 \
  --skip-list logs/knowledge/ingest_skipped.txt
```

(실제 CLI는 tools/metadata\_to\_knowledge.py 시그니처 확인 후 조정)

# 13. 사용자가 직접 실행할 명령어 레퍼런스

이 섹션은 **Claude/에이전트**가 수행하는 작업을 사용자가 직접 실행할 수 있도록 명령어를 모아 둔 cheatsheet. 모든 명령은 프로젝트 루트 ( c:\Users\Joon\Desktop\AI-pilot\ai-combat-sdk\ )에서 실행. Git Bash (Windows) 가정.

## 13.0 공통 설정

```
# 프로젝트 루트로 이동
cd /c/Users/Joon/Desktop/AI-pilot/ai-combat-sdk

# Python 3.14 고정 + UTF-8 필수 (Windows CP949 회피)
export PYTHONIOENCODING=utf-8
```



## 13.1 Stage ① COLLECT — 새 매치 수집

```
# 단일 BT vs 2 상대 × N rounds
PYTHONIOENCODING=utf-8 python -c "
import sys; sys.path.insert(0, '.')
from scripts.run_match import run_match
for opp in ['examples/aggressive.yaml', 'examples/defensive.yaml']:
    run_match(
        agent1='examples/adaptive_eagle_v9/adaptive_eagle_v9.yaml',
        agent2=opp, rounds=10, verbose=False,
        metadata_log='logs/matches/my_test_run')
"

# 695 풀 전체 검증 (병렬)
PYTHONIOENCODING=utf-8 python tools/adaptive_optimizer.py --validate \
    examples/adaptive_eagle_v9/adaptive_eagle_v9.yaml
```

## 13.2 Stage ② INGEST — CSV → matches.jsonl

```
# 단일 배치 ingest
PYTHONIOENCODING=utf-8 python tools/metadata_to_knowledge.py ingest \
    --collection-batch "my_tag_2026_04" \
    --agent-version "v9_test" \
    logs/matches/my_test_run/

# 28배치 bulk (shell loop - subprocess 인코딩 이슈 회피)
for batch in logs/matches/*/; do
    name=$(basename "$batch")
    PYTHONIOENCODING=utf-8 python tools/metadata_to_knowledge.py ingest \
        --collection-batch "legacy_${name}" \
        --agent-version "$name" \
        "$batch" 2>&1 | tail -1
done

# 평탄 CSV 디렉토리 (logs/metadata/)
PYTHONIOENCODING=utf-8 python tools/metadata_to_knowledge.py ingest \
    --collection-batch "legacy_metadata_flat" \
    --agent-version "mixed_legacy" \
    logs/metadata/

# 결과 확인
wc -l logs/knowledge/matches.jsonl
```

**주의:** metadata\_to\_knowledge.py 는 항상 logs/knowledge/matches.jsonl 에 **append**. 백업 없이 truncate 금지 (§11.15.2 #5 baseline 보존).

```
# 안전한 백업 패턴 (truncate 전 필수)
cp logs/knowledge/matches.jsonl logs/knowledge/matches.jsonl.bak_$(date +%Y%m%d_%H%M)
```

## 13.3 Stage ③ LEARN-EIM — 의도 분류기 학습

```
# ProtoNet 학습 (canonical)
PYTHONIOENCODING=utf-8 python tools/train_intent_model.py

# 결과
# → logs/knowledge/intent_model_trajectory.pt
# 또는 configurable path
```

## 13.4 Stage ⑤ MINE — 가설 자동 생성

```
# 전체 miner 실행
PYTHONIOENCODING=utf-8 python tools/hypothesis_miner.py

# 출력: logs/knowledge/hypothesis_queue.json
# 내부적으로 Miner 1 (rigid), 2 (outcome discriminator), 5 (node usage), 8 (tactical delta) 병렬
```

## 13.5 Stage ⑥ TRACK — 가설 검증

```
# 단일 가설 검증 (baseline vs experimental)
PYTHONIOENCODING=utf-8 python tools/hypothesis_tracker.py \
    --hypothesis-id H7 \
    --baseline examples/adaptive_eagle_v9/adaptive_eagle_v9.yaml \
    --experimental examples/adaptive_eagle_v9/adaptive_eagle_v9_H7.yaml \
    --rounds 2 \
    --opponents eagle1,eagle2,ace,viper1,golden,alpha2

# 결과
# → logs/knowledge/hypotheses.jsonl (CONFIRMED/REFUTED/INCONCLUSIVE)
```

## 13.6 Stage ⑧ PROFILE — 탐색적 분석

```
# 고수준 지표 리포트 (SAE, TIR, WPP, WCS, EIP, EVW)
PYTHONIOENCODING=utf-8 python tools/analyze_metadata.py \
    logs/matches/v9_TL_headOnAdaptive_20R/

# ACMI 시각화 보조
PYTHONIOENCODING=utf-8 python tools/analyze_acmi.py <acmi_file>
```

## 13.7 Stage ⑨ EVALUATE — 최종 검증

```
# Library 용 (evaluate.py)
PYTHONIOENCODING=utf-8 python -c "
from tools.evaluate import evaluate_agent
result = evaluate_agent(
    'examples/adaptive_eagle_v9/adaptive_eagle_v9.yaml',
    opponents=['eagle1', 'eagle2', 'ace', 'viper1', 'golden', 'alpha2'],
    rounds=5)
print(result)
"

# CLI 695 풀
PYTHONIOENCODING=utf-8 python tools/adaptive_optimizer.py --validate \
    examples/adaptive_eagle_v9/adaptive_eagle_v9.yaml
# → logs/full_pool_validation.json

# YAML 제출 전 정적 검증
PYTHONIOENCODING=utf-8 python tools/validate_agent.py \
    examples/adaptive_eagle_v9/adaptive_eagle_v9.yaml
```

## 13.8 WR/HP 분석 (매치 결과 집계)

```
# 한 디렉토리의 WR + HP 계산
PYTHONIOENCODING=utf-8 python << 'EOF'

import json, re, math
from pathlib import Path

def wilson(w, n, z=1.96):
    if n == 0: return (0, 0)
    p = w/n; denom = 1 + z*z/n
    centre = (p + z*z/(2*n))/denom
    half = z * math.sqrt(p*(1-p)/n + z*z/(4*n*n))/denom
    return (max(0, centre-half), min(1, centre+half))

D = Path('logs/matches/v9_TL_headOnAdaptive_20R')
pat = re.compile(r'\d{8}_\d{6}_(.+?)_vs_(aggressive|defensive)_round(\d+)')
res = {'aggressive': {'W':0, 'D':0, 'L':0, 'hp1':[], 'hp2':[]},
       'defensive': {'W':0, 'D':0, 'L':0, 'hp1':[], 'hp2':[]}}
for p in sorted(D.glob('*_result.json')):
    m = pat.match(p.stem.replace('_meta_result', '_meta'))
    if not m: continue
    bt, opp, _ = m.groups()
    d = json.load(open(p, encoding='utf-8'))
    hp1, hp2 = d.get('tree1_hp', 0), d.get('tree2_hp', 0)
    w = (d.get('winner') or '').lower()
    if bt in w or 'tree1' in w: o = 'W'
    elif opp in w or 'tree2' in w: o = 'L'
    else: o = 'D' if hp1 == hp2 else ('W' if hp1 > hp2 else 'L')
    res[opp][o] += 1
    res[opp]['hp1'].append(hp1); res[opp]['hp2'].append(hp2)

for opp, k in res.items():
    if not k['hp1']: continue
    n = k['W']+k['D']+k['L']
    avg1 = sum(k['hp1'])/len(k['hp1']); avg2 = sum(k['hp2'])/len(k['hp2'])
    wr = k['W']/n if n else 0
    lo, hi = wilson(k['W'], n)
    print(f'vs {opp:10s}  W/D/L={k["W"]:2d}/{k["D"]:2d}/{k["L"]:2d}  '
          f'HP {avg1:.1f}/{avg2:.1f}  WR={wr:.0%}  CI[{lo:.0%},{hi:.0%}]')
EOF
```

## 13.9 knowledge DB 탐색

```
# matches.jsonl 전체 행수
wc -l logs/knowledge/matches.jsonl

# 배치별 분포
PYTHONIOENCODING=utf-8 python -c "
import json
from collections import Counter
c = Counter()
for line in open('logs/knowledge/matches.jsonl', encoding='utf-8'):
    d = json.loads(line)
    c[d.get('tags', {}).get('collection_batch', 'unknown')] += 1
for k, v in c.most_common():
    print(f'{v:5d} {k}')
"

# 승/패/무승부 카테고리 분포
PYTHONIOENCODING=utf-8 python -c "
import json
from collections import Counter
c = Counter()
for line in open('logs/knowledge/matches.jsonl', encoding='utf-8'):
    d = json.loads(line)
    c[d.get('outcome', {}).get('category', '?')] += 1
for k, v in c.most_common():
    print(f'{v:5d} {k}')
"

# 특정 상대별 WR
PYTHONIOENCODING=utf-8 python -c "
import json
from collections import defaultdict
tots = defaultdict(lambda: [0,0,0]) # W,D,L
for line in open('logs/knowledge/matches.jsonl', encoding='utf-8'):
    d = json.loads(line)
    opp = d.get('opponent', {}).get('name', '?')
    cat = d.get('outcome', {}).get('category', '')
    if 'WIN' in cat: tots[opp][0] += 1
    elif 'DRAW' in cat: tots[opp][1] += 1
    elif 'LOSS' in cat: tots[opp][2] += 1
for opp, (w,dr,l) in sorted(tots.items(), key=lambda x: -(x[1][0])):
    n = w+dr+l
    if n: print(f'{opp:30s} W/D/L={w:3d}/{dr:3d}/{l:3d} WR={w/n:.0%} (n={n})')
"
```

## 13.10 단일 가설 A/B 수동 검증 (Stage ⑥ 대체)

10R × 2 상대 빠른 iteration (바로 로그에서 본 방식):

```
# 실험 디렉토리 생성
mkdir -p logs/matches/v9_experiment_$(date +%Y%m%d_%H%M%S)

# 실행
PYTHONIOENCODING=utf-8 python -c "
import sys, time; sys.path.insert(0, '.')
from scripts.run_match import run_match
BT = 'examples/adaptive_eagle_v9/adaptive_eagle_v9.yaml'
OUT = 'logs/matches/v9_experiment_YYYYMMDD_HHMMSS'
t0 = time.time()
for opp in ['examples/aggressive.yaml', 'examples/defensive.yaml']:
    run_match(agent1=BT, agent2=opp, rounds=10,
              verbose=False, metadata_log=OUT)
    print(f'OK {opp} at {time.time()-t0:.0f}s', file=sys.stderr)
" 2>&1 | tail -3
```

## 13.11 Commit 전 validation 체크리스트

```
# 1. YAML 문법 검증
PYTHONIOENCODING=utf-8 python tools/validate_agent.py \
    examples/adaptive_eagle_v9/adaptive_eagle_v9.yaml

# 2. baseline 보존 확인 (submissions/ 건드리지 않았나)
git status submissions/

# 3. 현재 matches.jsonl 행수 기록
wc -l logs/knowledge/matches.jsonl

# 4. 테스트 실행 (작아도 최소 한 번)
PYTHONIOENCODING=utf-8 python tools/test_suite.py
```

## 13.12 응급 복구 (baseline rollback)

```
# v9 YAML 원본 복구
cp submissions/adaptive_eagle/adaptive_eagle.yaml \
    examples/adaptive_eagle_v9/adaptive_eagle_v9.yaml

# matches.jsonl 복구 (가장 최근 .bak)
ls -t logs/knowledge/matches.jsonl.bak* | head -1 | \
    xargs -I{} cp {} logs/knowledge/matches.jsonl
```